



RTL Design Sherpa

RTL AMBA AXI5 Module Reference — Rev 0.90

July 2026

Table of Contents

1 AXI5 Master Read with Clock Gating.....	10
1.1 Overview.....	11
1.1.1 Key Features.....	11
1.2 Parameters.....	12
1.3 Ports.....	13
1.3.1 Clock and Reset.....	13
1.3.2 Clock Gating Configuration.....	13
1.3.3 FUB AXI5 Interface (Slave Side - Input).....	13
1.3.4 Master AXI5 Interface (Output Side).....	16
1.3.5 Clock Gating Status Outputs.....	16
1.4 Functionality.....	16
1.4.1 Clock Gating Operation.....	16
1.4.2 Idle Counter Configuration.....	16
1.4.3 Power Savings Analysis.....	17
1.5 Timing Diagrams.....	17
1.5.1 Clock Gating Activation.....	17
1.5.2 Clock Gating Wake-up.....	17
1.6 Usage Example.....	18
1.7 Design Notes.....	20
1.7.1 Clock Gating vs. Always-On.....	20
1.7.2 Configuration Strategies.....	20
1.7.3 Integration with System Power Management.....	20

1.7.4 Verification Considerations.....	21
1.8 Related Documentation.....	21
1.9 Navigation.....	21
2 AXI5 Master Read.....	21
2.1 Overview.....	21
2.1.1 Key Features.....	21
2.2 Parameters.....	23
2.3 Ports.....	24
2.3.1 Clock and Reset.....	24
2.3.2 FUB AXI5 Interface (Slave Side - Input).....	24
2.3.3 Master AXI5 Interface (Output Side).....	26
2.3.4 Status Outputs.....	26
2.4 Functionality.....	26
2.4.1 AXI5 Enhancements Over AXI4.....	26
2.4.2 Signal Packing Logic.....	27
2.5 Timing Diagrams.....	27
2.5.1 Basic Read Transaction.....	27
2.5.2 Chunked Read Transaction.....	27
2.5.3 Memory Tagging Extension (MTE).....	28
2.6 Usage Example.....	28
2.7 Design Notes.....	29
2.7.1 AXI5 vs AXI4 Differences.....	29
2.7.2 Memory Tagging Extension (MTE).....	30
2.7.3 Chunking Support.....	30

2.7.4 Feature Enable Strategy.....	30
2.8 Related Documentation.....	30
2.9 Navigation.....	31
3 AXI5 Master Write with Clock Gating.....	31
3.1 Overview.....	31
3.1.1 Key Features.....	31
3.2 Parameters.....	32
3.3 Ports.....	34
3.3.1 Clock and Reset.....	34
3.3.2 Clock Gating Configuration.....	34
3.3.3 FUB AXI5 Interface (Slave Side - Input).....	34
3.3.4 Master AXI5 Interface (Output Side).....	37
3.3.5 Clock Gating Status Outputs.....	37
3.4 Functionality.....	37
3.4.1 Clock Gating Operation.....	37
3.4.2 AXI5 Write-Specific Features.....	38
3.4.3 Idle Counter Configuration.....	38
3.5 Timing Diagrams.....	38
3.5.1 Clock Gating with Write Burst.....	38
3.5.2 Atomic Operation with Clock Gating.....	38
3.6 Usage Example.....	38
3.7 Design Notes.....	41
3.7.1 Write vs. Read Clock Gating.....	41
3.7.2 Write-Specific Gating Considerations.....	41

3.8 Related Documentation.....	41
3.9 Navigation.....	41
4 AXI5 Master Write.....	42
4.1 Overview.....	42
4.1.1 Key Features.....	42
4.2 Parameters.....	44
4.3 Ports.....	45
4.3.1 Clock and Reset.....	45
4.3.2 FUB AXI5 Interface (Slave Side - Input).....	45
4.3.3 Master AXI5 Interface (Output Side).....	47
4.3.4 Status Outputs.....	48
4.4 Functionality.....	48
4.4.1 AXI5 Enhancements Over AXI4.....	48
4.4.2 Atomic Operations.....	48
4.5 Timing Diagrams.....	49
4.5.1 Basic Write Transaction.....	49
4.5.2 Atomic Operation.....	49
4.5.3 Memory Tagging Extension (MTE).....	49
4.6 Usage Example.....	49
4.7 Design Notes.....	51
4.7.1 Atomic Operation Support.....	51
4.7.2 Memory Tagging Extension (MTE).....	51
4.7.3 Poison Support.....	52
4.7.4 Feature Enable Strategy.....	52

4.8 Related Documentation.....	52
4.9 Navigation.....	52
5 AXI5 Slave Read with Clock Gating.....	52
5.1 Overview.....	53
5.1.1 Key Features.....	53
5.2 Parameters.....	54
5.3 Ports.....	56
5.3.1 Clock and Reset.....	56
5.3.2 Clock Gating Configuration.....	56
5.3.3 Slave AXI5 Interface.....	57
5.3.4 FUB Interface.....	57
5.3.5 Clock Gating Status.....	57
5.4 Functionality.....	57
5.4.1 Clock Gating Behavior.....	57
5.5 Clock Gating Configuration.....	58
5.5.1 Idle Count Selection.....	58
5.6 Timing Diagrams.....	58
5.6.1 Clock Gating Sequence.....	58
5.7 Usage Example.....	58
5.8 Design Notes.....	60
5.8.1 Power Savings Estimation.....	60
5.8.2 Clock Gating Overhead.....	60
5.8.3 When to Use Clock Gating.....	60
5.8.4 Integration with Power Management.....	60

5.9 Related Documentation.....	61
5.10 Navigation.....	61
6 AXI5 Slave Read.....	61
6.1 Overview.....	61
6.1.1 Key Features.....	61
6.2 Parameters.....	62
6.3 Ports.....	64
6.3.1 Clock and Reset.....	64
6.3.2 Slave AXI5 Interface (External Master Input).....	64
6.3.3 FUB Interface (Backend Memory/Logic Output).....	66
6.3.4 Status Outputs.....	66
6.4 Functionality.....	67
6.5 Timing Diagrams.....	67
6.5.1 Basic Read Transaction.....	67
6.6 Usage Example.....	67
6.7 Design Notes.....	68
6.8 Related Documentation.....	68
6.9 Navigation.....	68
7 AXI5 Slave Write with Clock Gating.....	69
7.1 Overview.....	69
7.1.1 Key Features.....	69
7.2 Parameters.....	70
7.3 Ports.....	72
7.3.1 Clock and Reset.....	72

7.3.2 Clock Gating Configuration.....	72
7.3.3 Slave AXI5 Interface.....	72
7.3.4 FUB Interface.....	73
7.3.5 Clock Gating Status.....	73
7.4 Functionality.....	73
7.4.1 Clock Gating Behavior.....	73
7.5 Clock Gating Configuration.....	74
7.5.1 Idle Count Selection.....	74
7.6 Timing Diagrams.....	74
7.6.1 Clock Gating During Write Burst.....	74
7.7 Usage Example.....	74
7.8 Design Notes.....	76
7.8.1 Power Savings Estimation.....	76
7.8.2 Write-Specific Considerations.....	76
7.8.3 Clock Gating Overhead.....	76
7.8.4 When to Use Clock Gating.....	76
7.9 Related Documentation.....	76
7.10 Navigation.....	76
8 AXI5 Slave Write.....	77
8.1 Overview.....	77
8.1.1 Key Features.....	77
8.2 Parameters.....	79
8.3 Ports.....	80
8.3.1 Clock and Reset.....	80

8.3.2 Slave AXI5 Interface (External Master Input).....	81
8.3.3 FUB Interface (Backend Memory/Logic Output).....	83
8.3.4 Status Outputs.....	83
8.4 Functionality.....	83
8.5 Timing Diagrams.....	84
8.5.1 Basic Write Transaction.....	84
8.5.2 Write with Poison Indicator.....	84
8.6 Usage Example.....	84
8.7 Design Notes.....	85
8.8 Related Documentation.....	85
8.9 Navigation.....	85
9 AXI5 (Advanced eXtensible Interface - AMBA 5) Modules.....	86
9.1 Overview.....	86
9.2 AMBA4 vs AMBA5 Comparison.....	86
9.3 Module Categories.....	87
9.3.1 Master Components.....	87
9.3.2 Slave Components.....	87
9.3.3 Monitor Components.....	88
9.4 Key Features.....	88
9.4.1 AXI5 Protocol Support.....	88
9.4.2 Advanced Features.....	89
9.4.3 Power Management.....	89
9.4.4 Monitoring and Debug.....	89
9.5 Quick Start.....	89

9.5.1 Using AXI5 Read Master.....	89
9.5.2 Using AXI5 Write Master.....	90
9.5.3 Using Integrated Monitor Modules.....	91
9.6 Testing.....	92
9.7 Protocol Details.....	92
9.7.1 AXI5 Channel Overview.....	92
9.7.2 AXI5 Enhanced Signals.....	92
9.7.3 Burst Types.....	93
9.8 Design Notes.....	93
9.8.1 FUB Interface Pattern.....	93
9.8.2 Migration from AXI4.....	93
9.8.3 Monitor Integration.....	94
9.9 Performance Characteristics.....	94
9.10 Related Documentation.....	94
9.11 References.....	94
9.11.1 Specifications.....	94
9.11.2 Source Code.....	94
9.12 Navigation.....	95

1 AXI5 Master Read with Clock Gating

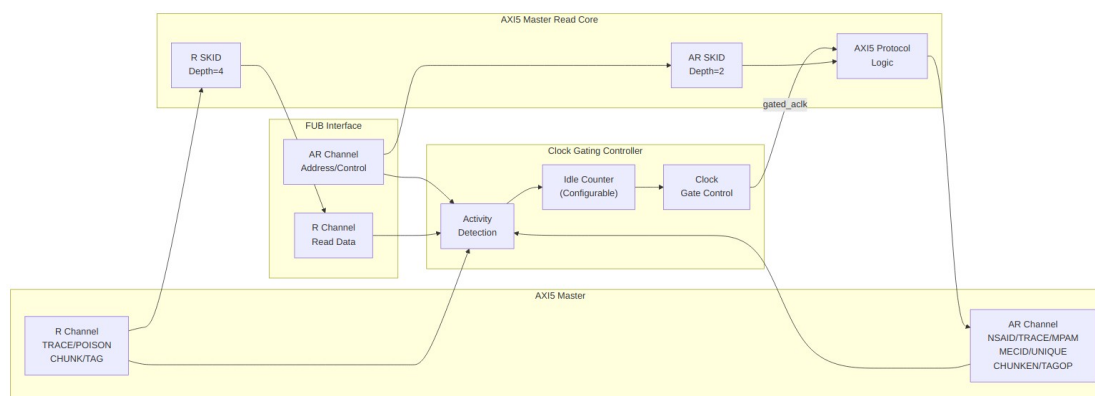
Module: axi5_master_rd_cg.sv **Location:** rtl/amba/axi5/ **Status:** Production Ready

1.1 Overview

The AXI5 Master Read with Clock Gating module wraps the standard `axi5_master_rd` core with intelligent clock gating for power optimization. It automatically gates the clock when no AXI activity is detected for a configurable number of idle cycles.

1.1.1 Key Features

- Full AMBA AXI5 protocol compliance (wraps `axi5_master_rd`)
- **ARNSAID**: Non-secure access identifier for security domains
- **ARTRACE**: Trace signal for debug and performance monitoring
- **ARMPAM**: Memory Partitioning and Monitoring (PartID + PMG)
- **ARMECID**: Memory Encryption Context ID for secure memory
- **ARUNIQUE**: Unique ID indicator for cache operations
- **ARCHUNKEN**: Read data chunking enable for partial data transfers
- **ARTAGOP**: Memory tag operation (MTE - Memory Tagging Extension)
- **RTRACE**: Read data trace signal
- **RPOISON**: Data poison indicator for corrupted data detection
- **RCHUNKV/RCHUNKNUM/RCHUNKSTRB**: Chunking control signals
- **RTAG/RTAGMATCH**: Memory tags and tag match response (MTE)
- **Automatic clock gating** based on activity detection
- **Configurable idle count** before clock gating activates
- **Power savings** during idle periods
- **Transparent operation** - no protocol changes
- **Status outputs** for clock gating state monitoring



Module Architecture

1.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR channel SKID buffer depth
SKID_DEPTH_R	int	4	R channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_WSTRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width (PartID + PMG)
AXI_MECID_WIDTH	int	16	Memory encryption context ID width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
AXI_CHUNKNUM_WIDTH	int	4	Chunk number width
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption context
ENABLE_UNIQUE	bit	1	Enable unique ID

Parameter	Type	Default	Description
			indicator
ENABLE_CHUNKING	bit	1	Enable data chunking
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter (max 2^N-1 cycles)

1.3 Ports

1.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock (ungated)
aresetn	1	Input	AXI active-low reset

1.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Clock gating enable (1=enable, 0=always active)
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating activates

1.3.3 FUB AXI5 Interface (Slave Side - Input)

1.3.3.1 AR Channel

Port	Width	Direction	Description
fub_axi_arid	IW	Input	Read address ID
fub_axi_araddr	AW	Input	Read address
fub_axi_arlen	8	Input	Burst length

Port	Width	Direction	Description
en			
fub_axi_ar_size	3	Input	Burst size
fub_axi_ar_burst	2	Input	Burst type
fub_axi_ar_lock	1	Input	Lock type
fub_axi_ar_cache	4	Input	Cache attributes
fub_axi_ar_prot	3	Input	Protection attributes
fub_axi_ar_qos	4	Input	Quality of Service
fub_axi_ar_user	UW	Input	User-defined signal
fub_axi_ar_valid	1	Input	Read address valid
fub_axi_ar_ready	1	Output	Read address ready (forced 0 when gating)

1.3.3.2 AXI5 AR Extensions

Port	Width	Direction	Description
fub_axi_ar_nsaid	AXI_NSAID_WIDTH	Input	Non-secure access ID
fub_axi_art_race	1	Input	Trace signal
fub_axi_ar_mpam	AXI_MPAM_WIDTH	Input	Memory partitioning/monitoring
fub_axi_ar_mecid	AXI_MECID_WIDTH	Input	Memory encryption context ID
fub_axi_ar_unique	1	Input	Unique ID indicator

Port	Width	Direction	Description
fub_axi_ar_chunken	1	Input	Chunking enable
fub_axi_art_agop	AXI_TAGOP_WIDTH	Input	Tag operation (MTE)

1.3.3.3 R Channel

Port	Width	Direction	Description
fub_axi_rid	IW	Output	Read data ID
fub_axi_rdata	DW	Output	Read data
fub_axi_rresp	2	Output	Read response
fub_axi_rlast	1	Output	Last transfer in burst
fub_axi_ruser	UW	Output	User-defined signal
fub_axi_rvalid	1	Output	Read data valid
fub_axi_rready	1	Input	Read data ready

1.3.3.4 AXI5 R Extensions

Port	Width	Direction	Description
fub_axi_rtrace	1	Output	Trace signal
fub_axi_rpoison	1	Output	Data poison indicator
fub_axi_rc_hunkv	1	Output	Chunk valid
fub_axi_rc_hunknum	AXI_CHUNK_NUM_WIDTH	Output	Chunk number
fub_axi_rc_hunkstrb	CHUNK_STRB_WIDTH	Output	Chunk strobe (128-bit granules)
fub_axi_rtag	TW	Output	Memory tags (NUM_TAGS × TAG_WIDTH)
fub_axi_rtag	1	Output	Tag match response

Port	Width	Direction	Description
gmatch			

1.3.4 Master AXI5 Interface (Output Side)

Same port list as FUB interface but with `m_axi_*` prefix and reversed directions.

1.3.5 Clock Gating Status Outputs

Port	Width	Direction	Description
cg_gating	1	Output	Clock gating active (1=clock gated, 0=clock running)
cg_idle	1	Output	Interface idle (1=no activity detected)

1.4 Functionality

1.4.1 Clock Gating Operation

The clock gating controller monitors activity on both user and AXI sides:

User-side activity detection:

```
user_valid = fub_axi_arvalid || fub_axi_rready || int_busy
```

AXI-side activity detection:

```
axi_valid = m_axi_arvalid || m_axi_rvalid
```

Clock gating state machine: 1. **ACTIVE:** Clock running, transactions in progress 2. **IDLE:** No activity detected, idle counter incrementing 3. **GATING:** Idle count reached, clock gated (power saving) 4. **WAKE:** Activity detected, clock ungating

Ready signal control during gating:

```
fub_axi_arready = cg_gating ? 1'b0 : int_arready;
m_axi_rready = cg_gating ? 1'b0 : int_rready;
```

1.4.2 Idle Counter Configuration

The `cfg_cg_idle_count` parameter sets how many consecutive idle cycles are required before clock gating activates:

cfg_cg_idle_count	Idle Cycles	Use Case
0	1 cycle	Aggressive power saving
1-3	2-4 cycles	Balanced (typical)
4-7	5-8 cycles	Low-latency systems
8-15	9-16 cycles	Minimize gating overhead

Recommendation: Start with `cfg_cg_idle_count = 3` (4 idle cycles) for most applications.

1.4.3 Power Savings Analysis

Clock gating effectiveness: - Dynamic power reduction proportional to idle time - Typical savings: 20-40% in bursty traffic scenarios - Maximum savings: >90% in mostly-idle systems

Trade-offs: - Wake latency: 1-2 clock cycles - Area increase: ~5% (clock gating logic) - Best suited for: Intermittent traffic patterns

1.5 Timing Diagrams

1.5.1 Clock Gating Activation

TODO: Wavedrom timing diagram showing:

- ACLK (ungated)
- GATED_ACLK (gated clock)
- user_valid, axi_valid activity
- Idle counter incrementing
- cg_idle assertion
- cg_gating activation
- Clock stops after idle count

1.5.2 Clock Gating Wake-up

TODO: Wavedrom timing diagram showing:

- ACLK (ungated)
 - GATED_ACLK resuming
 - Activity detection (arvalid assertion)
 - cg_gating deactivation
 - arready response after wake
 - Normal AXI transaction proceeds
-

1.6 Usage Example

```
axi5_master_rd_cg #(
    .AXI_ID_WIDTH      (8),
    .AXI_ADDR_WIDTH    (32),
    .AXI_DATA_WIDTH     (64),
    .AXI_USER_WIDTH     (4),
    .SKID_DEPTH_AR      (2),
    .SKID_DEPTH_R       (4),
    // Enable AXI5 features
    .ENABLE_NSaid       (1),
    .ENABLE_TRACE       (1),
    .ENABLE_MPAM        (1),
    .ENABLE_MECID       (1),
    .ENABLE_UNIQUE      (1),
    .ENABLE_CHUNKING    (1),
    .ENABLE_MTE         (1),
    .ENABLE_POISON      (1),
    // Clock gating configuration
    .CG_IDLE_COUNT_WIDTH (4)
) u_axi5_master_rd_cg (
    .aclk                (axi_clk),
    .aresetn              (axi_rst_n),

    // Clock gating configuration
    .cfg_cg_enable        (power_save_enable),
    .cfg_cg_idle_count    (4'd3), // Gate after 4 idle cycles

    // FUB interface (slave side)
    .fub_axi_arid         (fub_arid),
    .fub_axi_araddr       (fub_araddr),
    .fub_axi_arlen        (fub_arlen),
    .fub_axi_arsize       (fub_arsize),
    .fub_axi_arburst      (fub_arburst),
    .fub_axi_arlock       (fub_arlock),
    .fub_axi_arcache      (fub_arcache),
    .fub_axi_arprot       (fub_arprot),
    .fub_axi_arqos        (fub_arqos),
    .fub_axi_aruser       (fub_aruser),
    .fub_axi_arvalid      (fub_arvalid),
    .fub_axi_arready      (fub_arready),

    // AXI5 AR extensions
    .fub_axi_arnsaid      (fub_arnsaid),
    .fub_axi_artrace      (fub_artrace),
    .fub_axi_arpam        (fub_arpam),
    .fub_axi_armecid      (fub_armecid),
    .fub_axi_arunique     (fub_arunique),
```

```

.fub_axi_archunken (fub_archunken),
.fub_axi_artagop    (fub_artagop),

// R channel
.fub_axi_rid        (fub_rid),
.fub_axi_rdata      (fub_rdata),
.fub_axi_rresp      (fub_rresp),
.fub_axi_rlast      (fub_rlast),
.fub_axi_ruser      (fub_ruser),
.fub_axi_rvalid     (fub_rvalid),
.fub_axi_rready     (fub_rready),

// AXI5 R extensions
.fub_axi_rtrace     (fub_rtrace),
.fub_axi_rpoison    (fub_rpoison),
.fub_axi_rchunkv    (fub_rchunkv),
.fub_axi_rchunknum  (fub_rchunknum),
.fub_axi_rchunkstrb (fub_rchunkstrb),
.fub_axi_rtag       (fub_rtag),
.fub_axi_rtagmatch  (fub_rtagmatch),

// Master interface (output side)
.m_axi_arid         (m_axi_arid),
.m_axi_araddr       (m_axi_araddr),
// ... (connect all master signals similarly)

// Clock gating status
.cg_gating          (clock_gated),
.cg_idle            (interface_idle)
);

// Use status signals for power monitoring
always_ff @(posedge axi_clk or negedge axi_rst_n) begin
    if (!axi_rst_n)
        power_cycles_saved <= '0;
    else if (clock_gated)
        power_cycles_saved <= power_cycles_saved + 1;
end

```

1.7 Design Notes

1.7.1 Clock Gating vs. Always-On

Aspect	Always-On	Clock Gated
Power (idle)	100%	10-30%
Wake latency	0 cycles	1-2 cycles
Area overhead	0%	~5%
Best for	Continuous traffic	Bursty traffic

1.7.2 Configuration Strategies

Aggressive Power Saving:

```
.cfg_cg_enable      (1'b1),  
.cfg_cg_idle_count  (4'd0)  // Gate immediately after idle
```

Balanced (Recommended):

```
.cfg_cg_enable      (1'b1),  
.cfg_cg_idle_count  (4'd3)  // Gate after 4 idle cycles
```

Low-Latency:

```
.cfg_cg_enable      (1'b1),  
.cfg_cg_idle_count  (4'd7)  // Gate after 8 idle cycles
```

Disabled (Always-On):

```
.cfg_cg_enable      (1'b0),  
.cfg_cg_idle_count  (4'd0)  // Ignored when disabled
```

1.7.3 Integration with System Power Management

The clock gating status outputs can integrate with broader system power management:

```
// Aggregate clock gating status  
assign system_power_state = {  
    axi_rd_cg.cg_gating,  
    axi_wr_cg.cg_gating,  
    apb_cg.cg_gating  
    // ... other clock-gated modules  
};  
  
// Enter deep sleep when all interfaces gated  
assign deep_sleep_ok = &system_power_state;
```

1.7.4 Verification Considerations

When verifying clock-gated designs: 1. Test with `cfg_cg_enable = 0` first (baseline) 2. Enable clock gating and verify identical functional behavior 3. Check wake latency meets timing requirements 4. Verify no protocol violations during gating/ungating 5. Measure power savings in representative traffic scenarios

1.8 Related Documentation

- [AXI5 Master Read](#) - Non-gated base module
 - [AXI5 Master Write CG](#) - Write with clock gating
 - [AXI5 Master Read Monitor CG](#) - With monitoring + clock gating
 - [AMBA Clock Gate Controller](#) - Clock gating controller spec
-

1.9 Navigation

- [← Back to AXI5 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

2 AXI5 Master Read

Module: `axi5_master_rd.sv` **Location:** `rtl/amba/axi5/` **Status:** Production Ready

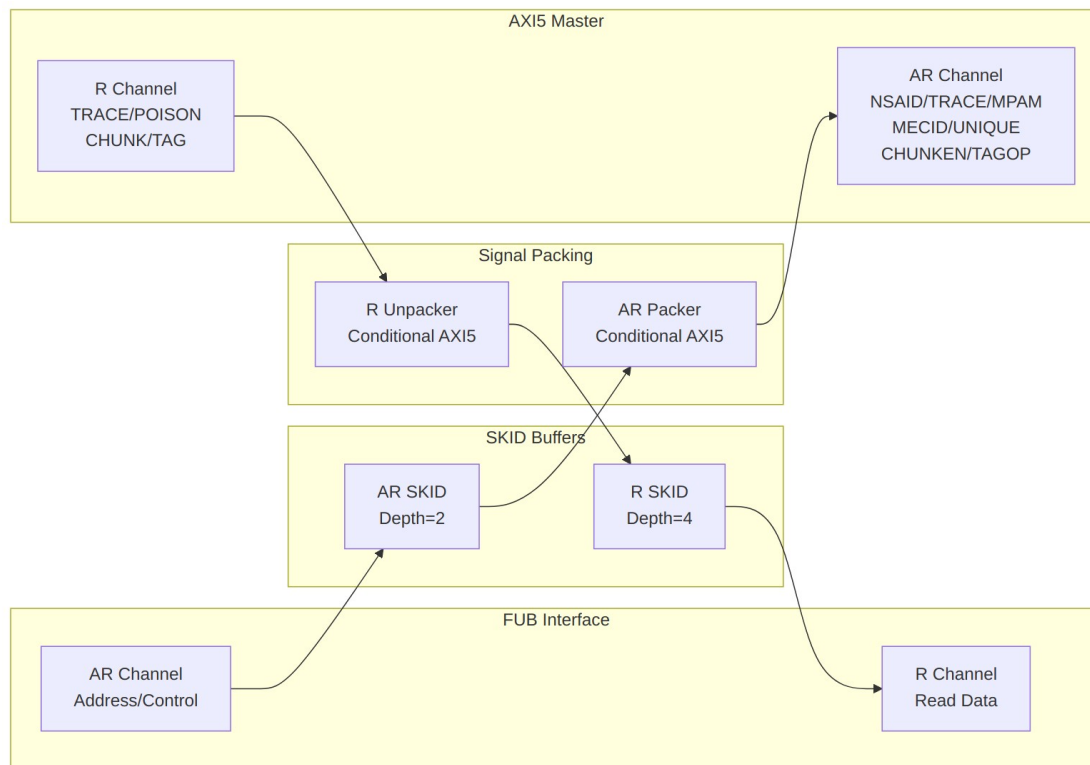
2.1 Overview

The AXI5 Master Read module implements a complete AMBA AXI5 master read interface with full AXI5 protocol support including all new AXI5 extensions. It provides SKID buffering for improved system performance and flow control.

2.1.1 Key Features

- Full AMBA AXI5 protocol compliance
- **ARNSAID:** Non-secure access identifier for security domains
- **ARTRACE:** Trace signal for debug and performance monitoring
- **ARMPAM:** Memory Partitioning and Monitoring (PartID + PMG)
- **ARMECID:** Memory Encryption Context ID for secure memory

- **ARUNIQUE:** Unique ID indicator for cache operations
 - **ARCHUNKEN:** Read data chunking enable for partial data transfers
 - **ARTAGOP:** Memory tag operation (MTE - Memory Tagging Extension)
 - **RTRACE:** Read data trace signal
 - **RPOISON:** Data poison indicator for corrupted data detection
 - **RCHUNKV/RCHUNKNUM/RCHUNKSTRB:** Chunking control signals
 - **RTAG/RTAGMATCH:** Memory tags and tag match response (MTE)
 - Configurable SKID buffer depths for AR and R channels
 - Busy signal for power management and clock gating
 - Removed ARREGION (deprecated in AXI5)
-



Module Architecture

2.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR channel SKID buffer depth
SKID_DEPTH_R	int	4	R channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_WSTRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width (PartID + PMG)
AXI_MECID_WIDTH	int	16	Memory encryption context ID width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
AXI_CHUNKNUM_WIDTH	int	4	Chunk number width
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption context
ENABLE_UNIQUE	bit	1	Enable unique ID indicator
ENABLE_CHUNKING	bit	1	Enable data chunking

Parameter	Type	Default	Description
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator

2.3 Ports

2.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

2.3.2 FUB AXI5 Interface (Slave Side - Input)

2.3.2.1 AR Channel

Port	Width	Direction	Description
fub_axi_arid	IW	Input	Read address ID
fub_axi_araddr	AW	Input	Read address
fub_axi_arlen	8	Input	Burst length
fub_axi_arsize	3	Input	Burst size
fub_axi_arburst	2	Input	Burst type
fub_axi_arlock	1	Input	Lock type
fub_axi_arcache	4	Input	Cache attributes
fub_axi_arprot	3	Input	Protection attributes
fub_axi_arqos	4	Input	Quality of Service
fub_axi_aruser	UW	Input	User-defined signal
fub_axi_arvali	1	Input	Read address

Port	Width	Direction	Description
d			valid
fub_axi_arready	1	Output	Read address ready

2.3.2.2 AXI5 AR Extensions

Port	Width	Direction	Description
fub_axi_arnsaid	AXI_NSAID_WIDTH	Input	Non-secure access ID
fub_axi_art race	1	Input	Trace signal
fub_axi_armpam	AXI_MPAM_WIDTH	Input	Memory partitioning/monitoring
fub_axi_armecid	AXI_MECID_WIDTH	Input	Memory encryption context ID
fub_axi_arunique	1	Input	Unique ID indicator
fub_axi_archunken	1	Input	Chunking enable
fub_axi_artagop	AXI_TAGOP_WIDTH	Input	Tag operation (MTE)

2.3.2.3 R Channel

Port	Width	Direction	Description
fub_axi_rid	IW	Output	Read data ID
fub_axi_rdata	DW	Output	Read data
fub_axi_rresp	2	Output	Read response
fub_axi_rlast	1	Output	Last transfer in burst
fub_axi_ruser	UW	Output	User-defined signal
fub_axi_rvalid	1	Output	Read data valid
fub_axi_rready	1	Input	Read data ready

2.3.2.4 AXI5 R Extensions

Port	Width	Direction	Description
fub_axi_rtr ace	1	Output	Trace signal
fub_axi_rp oison	1	Output	Data poison indicator
fub_axi_rc hunkv	1	Output	Chunk valid
fub_axi_rc hunknum	AXI_CHUNK NUM_WIDTH	Output	Chunk number
fub_axi_rc hunkstrb	CHUNK_STR B_WIDTH	Output	Chunk strobe (128-bit granules)
fub_axi_rta g	TW	Output	Memory tags (NUM_TAGS × TAG_WIDTH)
fub_axi_rta gmatch	1	Output	Tag match response

2.3.3 Master AXI5 Interface (Output Side)

Same port list as FUB interface but with m_axi_* prefix and reversed directions.

2.3.4 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy indicator for clock gating

2.4 Functionality

2.4.1 AXI5 Enhancements Over AXI4

Security and Isolation: - **ARNSAID:** Identifies security domain for non-secure accesses -
ARMECID: Provides encryption context for secure memory regions

Performance and Monitoring: - **ARTRACE:** Enables performance trace and debug capabilities -
ARMPAM: Supports memory bandwidth partitioning and QoS

Advanced Features: - **ARUNIQUE:** Indicates unique cache line access (no sharing) - **ARCHUNKEN:** Enables partial data transfer (chunking) - **ARTAGOP/RTAG/RTAGMATCH:** Memory Tagging Extension for security

Data Integrity: - **RPOISON:** Flags corrupted or invalid data - **RCHUNKV/RCHUNKNUM/RCHUNKSTRB:** Controls chunked data transfers

Deprecated: - **ARREGION:** Removed (not recommended for new designs)

2.4.2 Signal Packing Logic

The module uses dynamic packing based on feature enables:

```
// AR channel packed conditionally
if (ENABLE_NSaid)    → pack ARNSAID
if (ENABLE_TRACE)    → pack ARTRACE
if (ENABLE_MPAM)     → pack ARMPAM
if (ENABLE_MECID)    → pack ARMECID
if (ENABLE_UNIQUE)   → pack ARUNIQUE
if (ENABLE_CHUNKING) → pack ARCHUNKEN
if (ENABLE_MTE)      → pack ARTAGOP

// R channel unpacked conditionally
if (ENABLE_TRACE)    → unpack RTRACE
if (ENABLE_POISON)   → unpack RPOISON
if (ENABLE_CHUNKING) → unpack RCHUNKV/RCHUNKNUM/RCHUNKSTRB
if (ENABLE_MTE)      → unpack RTAG/RTAGMATCH
```

Disabled signals default to 0.

2.5 Timing Diagrams

2.5.1 Basic Read Transaction

TODO: Wavedrom timing diagram showing:

- ACLK
- ARID, ARADDR, ARLEN, ARSIZE
- ARVALID, ARREADY
- ARNSAID, ARTRACE, ARMPAM (AXI5 extensions)
- RID, RDATA, RRESP, RLAST
- RVALID, RREADY
- RTRACE, RPOISON (AXI5 extensions)

2.5.2 Chunked Read Transaction

TODO: Wavedrom timing diagram showing:

- ACLK
- ARCHUNKEN assertion

- RCHUNKV, RCHUNKNUM, RCHUNKSTRB
- Partial data transfer pattern

2.5.3 Memory Tagging Extension (MTE)

TODO: Wavedrom timing diagram showing:

- ARTAGOP encoding
 - RTAG delivery with data
 - RTAGMATCH response
-

2.6 Usage Example

```

axi5_master_rd #(
    .AXI_ID_WIDTH      (8),
    .AXI_ADDR_WIDTH    (32),
    .AXI_DATA_WIDTH     (64),
    .AXI_USER_WIDTH     (4),
    .SKID_DEPTH_AR      (2),
    .SKID_DEPTH_R       (4),
    // Enable AXI5 features
    .ENABLE_NSaid       (1),
    .ENABLE_TRACE       (1),
    .ENABLE_MPAM         (1),
    .ENABLE_MECID       (1),
    .ENABLE_UNIQUE      (1),
    .ENABLE_CHUNKING     (1),
    .ENABLE_MTE          (1),
    .ENABLE_POISON       (1)
) u_axi5_master_rd (
    .aclk                (axi_clk),
    .aresetn              (axi_rst_n),

    // FUB interface (slave side)
    .fub_axi_arid         (fub_arid),
    .fub_axi_araddr       (fub_araddr),
    .fub_axi_arlen        (fub_arlen),
    .fub_axi_arsize       (fub_arsize),
    .fub_axi_arburst      (fub_arburst),
    .fub_axi_arlock       (fub_arlock),
    .fub_axi_arcache      (fub_arcache),
    .fub_axi_arprot       (fub_arprot),
    .fub_axi_arqos        (fub_arqos),
    .fub_axi_aruser       (fub_aruser),
    .fub_axi_arvalid      (fub_arvalid),
    .fub_axi_arready      (fub_arready),

```

```

// AXI5 AR extensions
.fub_axi_arnsaid      (fub_arnsaid),
.fub_axi_artrace      (fub_artrace),
.fub_axi_arpam        (fub_arpam),
.fub_axi_armecid      (fub_armecid),
.fub_axi_arunique     (fub_arunique),
.fub_axi_archunken    (fub_archunken),
.fub_axi_artagop      (fub_artagop),

// R channel
.fub_axi_rid          (fub_rid),
.fub_axi_rdata        (fub_rdata),
.fub_axi_rresp        (fub_rresp),
.fub_axi_rlast        (fub_rlast),
.fub_axi_ruser        (fub_ruser),
.fub_axi_rvalid       (fub_rvalid),
.fub_axi_rready       (fub_rready),

// AXI5 R extensions
.fub_axi_rtrace       (fub_rtrace),
.fub_axi_rpoison      (fub_rpoison),
.fub_axi_rchunkv      (fub_rchunkv),
.fub_axi_rchunknum    (fub_rchunknum),
.fub_axi_rchunkstrb   (fub_rchunkstrb),
.fub_axi_rtag         (fub_rtag),
.fub_axi_rtagmatch    (fub_rtagmatch),

// Master interface (output side)
.m_axi_arid           (m_axi_arid),
.m_axi_araddr         (m_axi_araddr),
// ... (connect all master signals similarly)

// Status
.busy                 (master_rd_busy)
);

```

2.7 Design Notes

2.7.1 AXI5 vs AXI4 Differences

Feature	AXI4	AXI5
Security domains	None	NSAID
Trace support	None	TRACE signals

Feature	AXI4	AXI5
Memory partitioning	None	MPAM
Encryption context	None	MECID
Unique access	None	UNIQUE
Data chunking	None	CHUNKING
Memory tagging	None	MTE (TAG/TAGOP)
Poison indication	None	POISON
Region field	ARREGION	Deprecated

2.7.2 Memory Tagging Extension (MTE)

When `ENABLE_MTE=1`: - **ARTAGOP**: Specifies tag operation (Match/Insert/Fetch) - **RTAG**: Delivers memory tags (1 tag per 16 bytes of data) - **RTAGMATCH**: Indicates tag match result - **NUM_TAGS**: Calculated as $(DATA_WIDTH / 128)$, minimum 1

Example for 128-bit data bus: - $NUM_TAGS = 1 - TW = AXI_TAG_WIDTH \times 1 = 4$ bits

2.7.3 Chunking Support

When `ENABLE_CHUNKING=1`: - **ARCHUNKEN**: Master requests chunked response - **RCHUNKV**: Indicates chunk is valid - **RCHUNKNUM**: Identifies chunk within transaction - **RCHUNKSTRB**: Byte strobes at 128-bit granularity

2.7.4 Feature Enable Strategy

Disable unused features to reduce area:

```
.ENABLE_NSaid      (0), // No security domains
.ENABLE_TRACE      (0), // No trace capability
.ENABLE_MPAM       (0), // No memory partitioning
.ENABLE_MECID      (0), // No encryption
.ENABLE_UNIQUE     (0), // No unique access
.ENABLE_CHUNKING   (0), // No chunking
.ENABLE_MTE        (0), // No memory tagging
.ENABLE_POISON     (0)  // No poison detection
```

2.8 Related Documentation

- [AXI5 Master Write](#) - Master write interface
- [AXI5 Slave Read](#) - Slave read interface
- [AXI5 Master Read CG](#) - Clock-gated variant

- [AXI5 Master Read Monitor](#) - With integrated monitoring
 - [AXI4 Master Read](#) - AXI4 version for comparison
-

2.9 Navigation

- [← Back to AXI5 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

3 AXI5 Master Write with Clock Gating

Module: `axi5_master_wr_cg.sv` **Location:** `rtl/amba/axi5/` **Status:** Production Ready

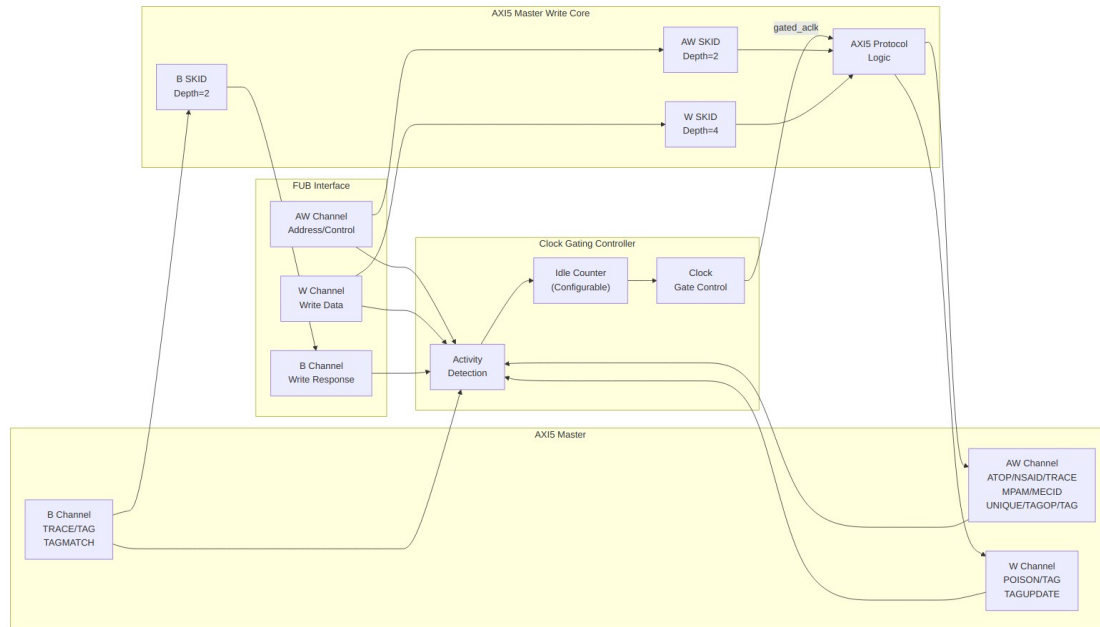
3.1 Overview

The AXI5 Master Write with Clock Gating module wraps the standard `axi5_master_wr` core with intelligent clock gating for power optimization. It automatically gates the clock when no AXI write activity is detected for a configurable number of idle cycles.

3.1.1 Key Features

- Full AMBA AXI5 protocol compliance (wraps `axi5_master_wr`)
- **AWATOP:** Atomic operation support (compare-and-swap, atomic operations)
- **AWNSAID:** Non-secure access identifier for security domains
- **AWTRACE:** Trace signal for debug and performance monitoring
- **AWMPAM:** Memory Partitioning and Monitoring (PartID + PMG)
- **AWMECID:** Memory Encryption Context ID for secure memory
- **AWUNIQUE:** Unique ID indicator for cache operations
- **AWTAGOP:** Memory tag operation (MTE - Memory Tagging Extension)
- **AWTAG:** Memory tags on address channel
- **WPOISON:** Write data poison indicator for corrupted data detection
- **WTAG/WTAGUPDATE:** Memory tags and tag update control (MTE)
- **BTRACE/BTAG/BTAGMATCH:** Response trace and tag signals
- **Automatic clock gating** based on activity detection
- **Configurable idle count** before clock gating activates

- **Power savings** during idle periods
- **Transparent operation** - no protocol changes
- **Status outputs** for clock gating state monitoring



Module Architecture

3.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel SKID buffer depth
SKID_DEPTH_W	int	4	W channel SKID buffer depth
SKID_DEPTH_B	int	2	B channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width

Parameter	Type	Default	Description
AXI_WSTRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
AXI_ATOP_WIDTH	int	6	Atomic operation width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width (PartID + PMG)
AXI_MECID_WIDTH	int	16	Memory encryption context ID width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
ENABLE_ATOMIC	bit	1	Enable atomic operations
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption context
ENABLE_UNIQUE	bit	1	Enable unique ID indicator
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter (max 2^N-1 cycles)

3.3 Ports

3.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock (ungated)
aresetn	1	Input	AXI active-low reset

3.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Clock gating enable (1=enable, 0=always active)
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating activates

3.3.3 FUB AXI5 Interface (Slave Side - Input)

3.3.3.1 AW Channel

Port	Width	Direction	Description
fub_axi_aw_id	IW	Input	Write address ID
fub_axi_aw_addr	AW	Input	Write address
fub_axi_aw_len	8	Input	Burst length
fub_axi_aw_size	3	Input	Burst size
fub_axi_aw_burst	2	Input	Burst type
fub_axi_aw_lock	1	Input	Lock type
fub_axi_aw_cache	4	Input	Cache attributes
fub_axi_aw	3	Input	Protection attributes

Port	Width	Direction	Description
prot			
fub_axi_aw_qos	4	Input	Quality of Service
fub_axi_aw_user	UW	Input	User-defined signal
fub_axi_aw_valid	1	Input	Write address valid
fub_axi_aw_ready	1	Output	Write address ready (forced 0 when gating)

3.3.3.2 AXI5 AW Extensions

Port	Width	Direction	Description
fub_axi_aw_atop	AXI_ATOP_WIDTH	Input	Atomic operation
fub_axi_aw_nsaid	AXI_NSAID_WIDTH	Input	Non-secure access ID
fub_axi_aw_trace	1	Input	Trace signal
fub_axi_aw_mpam	AXI_MPAM_WIDTH	Input	Memory partitioning/monitoring
fub_axi_aw_mecid	AXI_MECID_WIDTH	Input	Memory encryption context ID
fub_axi_aw_unique	1	Input	Unique ID indicator
fub_axi_aw_tagop	AXI_TAGOP_WIDTH	Input	Tag operation (MTE)
fub_axi_aw_tag	TW	Input	Memory tags on address channel

3.3.3.3 W Channel

Port	Width	Direction	Description
fub_axi_w_data	DW	Input	Write data
fub_axi_ws	SW	Input	Write strobes

Port	Width	Direction	Description
trb			
fub_axi_wlast	1	Input	Last transfer in burst
fub_axi_wuser	UW	Input	User-defined signal
fub_axi_wvalid	1	Input	Write data valid
fub_axi_wready	1	Output	Write data ready (forced 0 when gating)

3.3.3.4 AXI5 W Extensions

Port	Width	Direction	Description
fub_axi_wpoison	1	Input	Write data poison indicator
fub_axi_wtag	TW	Input	Memory tags on write data
fub_axi_wtagupdate	NUM_TAGS	Input	Tag update control (per 16B)

3.3.3.5 B Channel

Port	Width	Direction	Description
fub_axi_bid	IW	Output	Write response ID
fub_axi_bresp	2	Output	Write response
fub_axi_buser	UW	Output	User-defined signal
fub_axi_bvalid	1	Output	Write response valid
fub_axi_bready	1	Input	Write response ready

3.3.3.6 AXI5 B Extensions

Port	Width	Direction	Description
fub_axi_btrace	1	Output	Trace signal
fub_axi_btag	TW	Output	Memory tags in response
fub_axi_btagmatch	1	Output	Tag match result

3.3.4 Master AXI5 Interface (Output Side)

Same port list as FUB interface but with m_axi_* prefix and reversed directions.

3.3.5 Clock Gating Status Outputs

Port	Width	Direction	Description
cg_gating	1	Output	Clock gating active (1=clock gated, 0=clock running)
cg_idle	1	Output	Interface idle (1=no activity detected)

3.4 Functionality

3.4.1 Clock Gating Operation

The clock gating controller monitors activity on both user and AXI sides for all three write channels:

User-side activity detection:

```
user_valid = fub_axi_awvalid || fub_axi_wvalid || fub_axi_bready ||  
int_busy
```

AXI-side activity detection:

```
axi_valid = m_axi_awvalid || m_axi_wvalid || m_axi_bvalid
```

Ready signal control during gating:

```
fub_axi_awready = cg_gating ? 1'b0 : int_awready;  
fub_axi_wready = cg_gating ? 1'b0 : int_wready;  
m_axi_bready = cg_gating ? 1'b0 : int_bready;
```

All three write channels must be idle before clock gating activates, ensuring no partial writes.

3.4.2 AXI5 Write-Specific Features

Atomic Operations (AWATOP): When `ENABLE_ATOMIC=1`, supports: - AtomicStore - AtomicLoad - AtomicSwap - AtomicCompare - Atomic arithmetic operations

Memory Tagging on Write (MTE): When `ENABLE_MTE=1`: - **AWTAGOP**: Specifies tag operation for write address - **AWTAG**: Memory tags sent with address - **WTAG**: Memory tags sent with each data beat - **WTAGUPDATE**: Controls which tags to update (per 16 bytes) - **BTAG**: Memory tags returned in response - **BTAGMATCH**: Tag match result

Write Poison (WPOISON): When `ENABLE_POISON=1`: - Indicates corrupted or invalid write data - Propagated to memory/cache - May trigger error response (SLVERR)

3.4.3 Idle Counter Configuration

Same as read variant - see [AXI5 Master Read CG](#).

3.5 Timing Diagrams

3.5.1 Clock Gating with Write Burst

TOD0: Wavedrom timing diagram showing:

- ACLK (ungated)
- GATED_ACLK (gated clock)
- AW channel: AWVALID, AWREADY
- W channel: WDATA, WLAST, WVALID, WREADY
- B channel: BVALID, BREADY
- Idle counter after BREADY
- cg_gating activation after idle count

3.5.2 Atomic Operation with Clock Gating

TOD0: Wavedrom timing diagram showing:

- AWATOP encoding
- Atomic write sequence
- BTAG response
- Clock gating after completion

3.6 Usage Example

```
axi5_master_wr_cg #(
    .AXI_ID_WIDTH      (8),
    .AXI_ADDR_WIDTH    (32),
```

```

.AXI_DATA_WIDTH      (64),
.AXI_USER_WIDTH      (4),
.SKID_DEPTH_AW       (2),
.SKID_DEPTH_W        (4),
.SKID_DEPTH_B        (2),
// Enable AXI5 features
.ENABLE_ATOMIC       (1),
.ENABLE_NSaid        (1),
.ENABLE_TRACE        (1),
.ENABLE_MPAM         (1),
.ENABLE_MECID        (1),
.ENABLE_UNIQUE       (1),
.ENABLE_MTE          (1),
.ENABLE_POISON       (1),
// Clock gating configuration
.CG_IDLE_COUNT_WIDTH (4)
) u_axi5_master_wr_cg (
    .aclk              (axi_clk),
    .aresetn           (axi_rst_n),

    // Clock gating configuration
    .cfg_cg_enable     (power_save_enable),
    .cfg_cg_idle_count (4'd3), // Gate after 4 idle cycles

    // FUB interface (slave side)
    .fub_axi_awid      (fub_awid),
    .fub_axi_awaddr    (fub_awaddr),
    .fub_axi_awlen     (fub_awlen),
    .fub_axi_awsz      (fub_awsz),
    .fub_axi_awburst   (fub_awburst),
    .fub_axi_awlock    (fub_awlock),
    .fub_axi_awcache   (fub_awcache),
    .fub_axi_awprot    (fub_awprot),
    .fub_axi_awqos     (fub_awqos),
    .fub_axi_awuser    (fub_awuser),
    .fub_axi_awvalid   (fub_awvalid),
    .fub_axi_awready   (fub_awready),

    // AXI5 AW extensions
    .fub_axi_awatop    (fub_awatop),
    .fub_axi_awnsaid   (fub_awnsaid),
    .fub_axi_awtrace   (fub_awtrace),
    .fub_axi_awmpam    (fub_awmpam),
    .fub_axi_awmecid   (fub_awmecid),
    .fub_axi_awunique   (fub_awunique),
    .fub_axi_awtagop   (fub_awtagop),
    .fub_axi_awtag     (fub_awtag),

```

```

// W channel
.fub_axi_wdata      (fub_wdata),
.fub_axi_wstrb      (fub_wstrb),
.fub_axi_wlast      (fub_wlast),
.fub_axi_wuser      (fub_wuser),
.fub_axi_wvalid      (fub_wvalid),
.fub_axi_wready      (fub_wready),

// AXI5 W extensions
.fub_axi_wpoison     (fub_wpoison),
.fub_axi_wtag        (fub_wtag),
.fub_axi_wtagupdate  (fub_wtagupdate),

// B channel
.fub_axi_bid         (fub_bid),
.fub_axi_bresp       (fub_bresp),
.fub_axi_buser       (fub_buser),
.fub_axi_bvalid      (fub_bvalid),
.fub_axi_bready      (fub_bready),

// AXI5 B extensions
.fub_axi_btrace      (fub_btrace),
.fub_axi_btag        (fub_btag),
.fub_axi_btagmatch   (fub_btagmatch),

// Master interface (output side)
.m_axi_awid          (m_axi_awid),
.m_axi_awaddr        (m_axi_awaddr),
// ... (connect all master signals similarly)

// Clock gating status
.cg_gating           (clock_gated),
.cg_idle             (interface_idle)
);

// Power monitoring
always_ff @(posedge axi_clk or negedge axi_rst_n) begin
    if (!axi_rst_n) begin
        write_count <= '0;
        power_cycles_saved <= '0;
    end else begin
        if (fub_awvalid && fub_awready)
            write_count <= write_count + 1;

        if (clock_gated)

```



```

        power_cycles_saved <= power_cycles_saved + 1;
    end
end

```

3.7 Design Notes

3.7.1 Write vs. Read Clock Gating

Aspect	Read (AR/R)	Write (AW/W/B)
Channels	2 (AR, R)	3 (AW, W, B)
Activity detection	2 valids	3 valids
Idle condition	Both channels idle	All 3 channels idle
Typical idle %	Higher (read-mostly traffic)	Lower (write completion delays)
Power savings	30-50%	20-40%

3.7.2 Write-Specific Gating Considerations

Write data buffering: - Clock may gate after AW but before W (if W delayed) - SKID buffers ensure no data loss - Gate activation waits for W channel idle

Response latency: - B channel responses may delay gating - Typical: B arrives many cycles after W - Idle counter starts after BREADY

Atomic operations: - Longer latency than normal writes - Atomic read-modify-write sequence - Idle detection after atomic completion

3.8 Related Documentation

- [AXI5 Master Write](#) - Non-gated base module
 - [AXI5 Master Read CG](#) - Read with clock gating
 - [AXI5 Master Write Monitor CG](#) - With monitoring + clock gating
 - [AMBA Clock Gate Controller](#) - Clock gating controller spec
-

3.9 Navigation

- [← Back to AXI5 Index](#)

- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

4 AXI5 Master Write

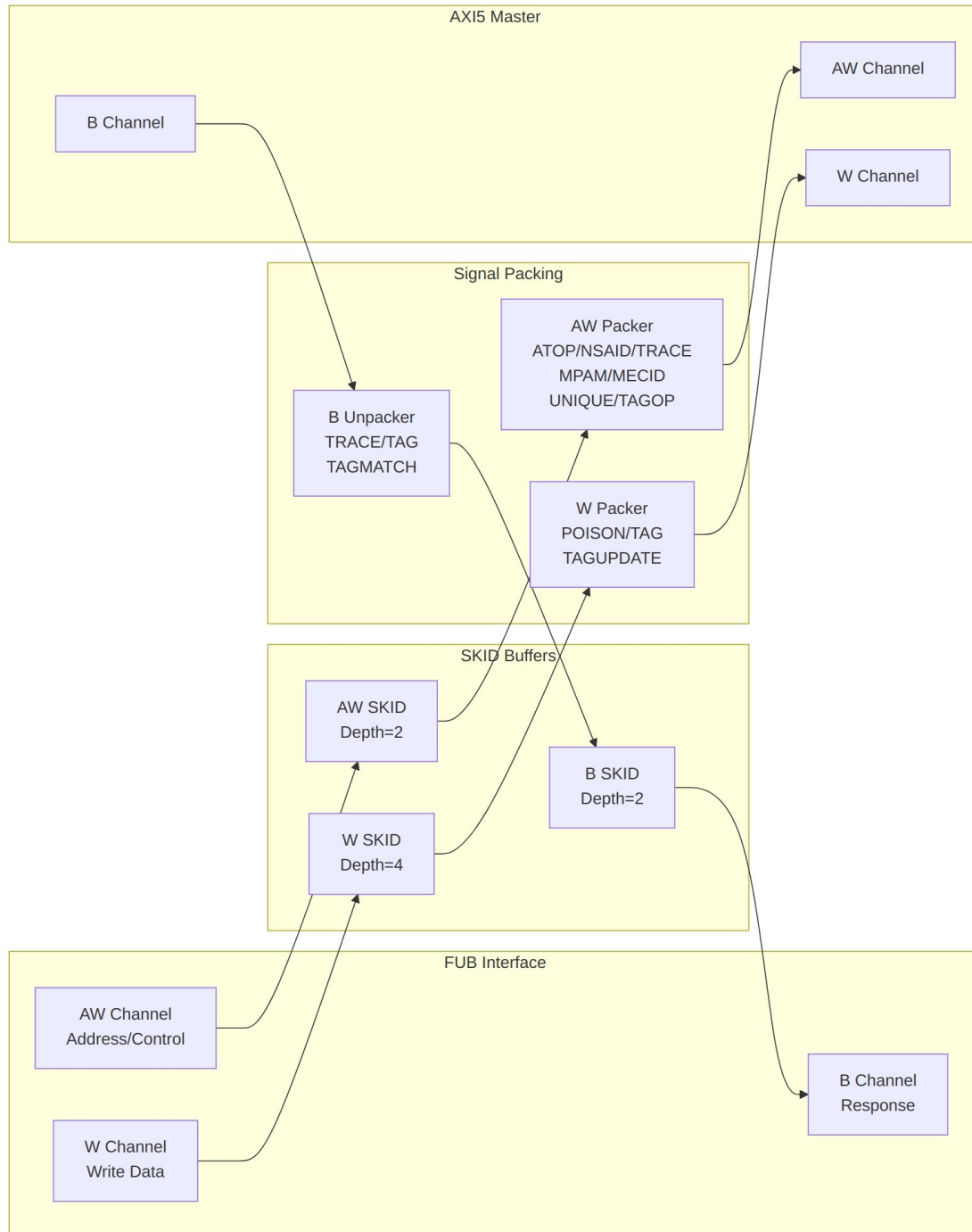
Module: axi5_master_wr.sv **Location:** rtl/amba/axi5/ **Status:** Production Ready

4.1 Overview

The AXI5 Master Write module implements a complete AMBA AXI5 master write interface with full AXI5 protocol support including atomic operations and all new AXI5 extensions. It provides SKID buffering for AW, W, and B channels for improved system performance.

4.1.1 Key Features

- Full AMBA AXI5 protocol compliance
 - **AWATOP:** Atomic transaction operation type (Compare/Swap, Fetch/Add, etc.)
 - **AWNSAID:** Non-secure access identifier for security domains
 - **AWTRACE:** Trace signal for debug and performance monitoring
 - **AWMPAM:** Memory Partitioning and Monitoring (PartID + PMG)
 - **AWMECID:** Memory Encryption Context ID for secure memory
 - **AWUNIQUE:** Unique ID indicator for cache operations
 - **AWTAGOP:** Memory tag operation (MTE - Memory Tagging Extension)
 - **AWTAG:** Memory tags for address
 - **WPOISON:** Write data poison indicator for error injection
 - **WTAG:** Write data memory tags (MTE)
 - **WTAGUPDATE:** Tag update mask for selective tag writing
 - **BTRACE:** Response trace signal
 - **BTAG/BTAGMATCH:** Response memory tags and tag match
 - Configurable SKID buffer depths for AW, W, and B channels
 - Busy signal for power management and clock gating
 - Removed AWREGION (deprecated in AXI5)
-



Module Architecture

4.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel SKID buffer depth
SKID_DEPTH_W	int	4	W channel SKID buffer depth
SKID_DEPTH_B	int	2	B channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_WSTRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
AXI_ATOP_WIDTH	int	6	Atomic operation width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width (PartID + PMG)
AXI_MECID_WIDTH	int	16	Memory encryption context ID width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
ENABLE_ATOMIC	bit	1	Enable atomic operations
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption context

Parameter	Type	Default	Description
ENABLE_UNIQUE	bit	1	Enable unique ID indicator
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator

4.3 Ports

4.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

4.3.2 FUB AXI5 Interface (Slave Side - Input)

4.3.2.1 AW Channel

Port	Width	Direction	Description
fub_axi_awid	IW	Input	Write address ID
fub_axi_awaddr	AW	Input	Write address
fub_axi_awlen	8	Input	Burst length
fub_axi_awsize	3	Input	Burst size
fub_axi_awburst	2	Input	Burst type
fub_axi_awlock	1	Input	Lock type
fub_axi_awcache	4	Input	Cache attributes
fub_axi_awprot	3	Input	Protection attributes
fub_axi_awqos	4	Input	Quality of

Port	Width	Direction	Description
			Service
fub_axi_awuser	UW	Input	User-defined signal
fub_axi_awvalid	1	Input	Write address valid
fub_axi_awready	1	Output	Write address ready

4.3.2.2 AXI5 AW Extensions

Port	Width	Direction	Description
fub_axi_awatop	AXI_ATOP_WIDTH	Input	Atomic operation type
fub_axi_awnsaid	AXI_NSAID_WIDTH	Input	Non-secure access ID
fub_axi_awtrace	1	Input	Trace signal
fub_axi_awmpam	AXI_MPAM_WIDTH	Input	Memory partitioning/monitoring
fub_axi_awmecid	AXI_MECID_WIDTH	Input	Memory encryption context ID
fub_axi_awunique	1	Input	Unique ID indicator
fub_axi_awtagop	AXI_TAGOP_WIDTH	Input	Tag operation (MTE)
fub_axi_awtag	TW	Input	Address memory tags

4.3.2.3 W Channel

Port	Width	Direction	Description
fub_axi_wdata	DW	Input	Write data
fub_axi_wstrb	SW	Input	Write strobes
fub_axi_wlast	1	Input	Last transfer in burst
fub_axi_wuser	UW	Input	User-defined

Port	Width	Direction	Description
fub_axi_wvalid	1	Input	Write data valid
fub_axi_wready	1	Output	Write data ready

4.3.2.4 AXI5 W Extensions

Port	Width	Direction	Description
fub_axi_wpoison	1	Input	Data poison indicator
fub_axi_wtag	TW	Input	Write data tags
fub_axi_wtagupdate	NUM_TAGS	Input	Tag update mask

4.3.2.5 B Channel

Port	Width	Direction	Description
fub_axi_bid	IW	Output	Response ID
fub_axi_bresp	2	Output	Write response
fub_axi_buser	UW	Output	User-defined signal
fub_axi_bvalid	1	Output	Write response valid
fub_axi_bready	1	Input	Write response ready

4.3.2.6 AXI5 B Extensions

Port	Width	Direction	Description
fub_axi_btrace	1	Output	Response trace
fub_axi_btag	TW	Output	Response tags
fub_axi_btagmatch	1	Output	Tag match response

4.3.3 Master AXI5 Interface (Output Side)

Same port list as FUB interface but with m_axi_* prefix and reversed directions.

4.3.4 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy indicator for clock gating

4.4 Functionality

4.4.1 AXI5 Enhancements Over AXI4

Atomic Operations: - **AWATOP:** Supports atomic Compare/Swap, Fetch/Add, Fetch/And, etc. - Enables lock-free synchronization primitives

Security and Isolation: - **AWNSAID:** Identifies security domain for non-secure accesses - **AWMECID:** Provides encryption context for secure memory regions

Performance and Monitoring: - **AWTRACE:** Enables performance trace and debug capabilities - **AWMPAM:** Supports memory bandwidth partitioning and QoS

Advanced Features: - **AWUNIQUE:** Indicates unique cache line access (no sharing) - **AWTAGOP/AWTAG/WTAG:** Memory Tagging Extension for security - **WTAGUPDATE:** Selective tag update mask

Data Integrity: - **WPOISON:** Indicates corrupted or test data - **BTRACE/BTAG/BTAGMATCH:** Response with trace and tag information

Deprecated: - **AWREGION:** Removed (not recommended for new designs)

4.4.2 Atomic Operations

AWATOP encoding (6 bits):

[5:4] - Atomic type:

- 00 = None
- 01 = Swap
- 10 = Compare
- 11 = Fetch-op

[3:0] - Operation (when Fetch-op):

- 0000 = ADD
- 0001 = CLR (AND-NOT)
- 0010 = EOR (XOR)
- 0011 = SET (OR)
- 0100 = SMAX
- 0101 = SMIN

0110 = UMAX
0111 = UMIN

4.5 Timing Diagrams

4.5.1 Basic Write Transaction

TODO: Wavedrom timing diagram showing:

- ACLK
- AWID, AWADDR, AWLEN, AWSIZE
- AWVALID, AWREADY
- AWATOP, AWNSAID, AWTRACE (AXI5 extensions)
- WDATA, WSTRB, WLAST
- WVALID, WREADY
- WPOISON, WTAG (AXI5 extensions)
- BID, BRESP
- BVALID, BREADY
- BTRACE, BTAG (AXI5 extensions)

4.5.2 Atomic Operation

TODO: Wavedrom timing diagram showing:

- AWATOP encoding for atomic operation
- Write data for atomic operand
- Response with atomic result

4.5.3 Memory Tagging Extension (MTE)

TODO: Wavedrom timing diagram showing:

- AWTAGOP encoding
 - AWTAG delivery with address
 - WTAG delivery with data
 - WTAGUPDATE mask
 - BTAGMATCH response
-

4.6 Usage Example

```
axi5_master_wr #(
    .AXI_ID_WIDTH      (8),
    .AXI_ADDR_WIDTH    (32),
    .AXI_DATA_WIDTH     (64),
    .AXI_USER_WIDTH     (4),
    .SKID_DEPTH_AW      (2),
    .SKID_DEPTH_W        (4),
    .SKID_DEPTH_B        (2),
```

```

// Enable AXI5 features
.ENABLE_ATOMIC      (1),
.ENABLE_NSaid       (1),
.ENABLE_TRACE       (1),
.ENABLE_MPAM        (1),
.ENABLE_MECID       (1),
.ENABLE_UNIQUE      (1),
.ENABLE_MTE         (1),
.ENABLE_POISON      (1)
) u_axi5_master_wr (
    .aclk             (axi_clk),
    .aresetn          (axi_rst_n),

// FUB interface (slave side)
.fub_axi_awid        (fub_awid),
.fub_axi_awaddr      (fub_awaddr),
.fub_axi_awlen       (fub_awlen),
.fub_axi_awsiz       (fub_awsiz),
.fub_axi_awburst     (fub_awburst),
.fub_axi_awlock      (fub_awlock),
.fub_axi_awcache     (fub_awcache),
.fub_axi_awprot      (fub_awprot),
.fub_axi_awqos       (fub_awqos),
.fub_axi_awuser      (fub_awuser),
.fub_axi_awvalid     (fub_awvalid),
.fub_axi_awready     (fub_awready),

// AXI5 AW extensions
.fub_axi_awatop      (fub_awatop),
.fub_axi_awnsaid     (fub_awnsaid),
.fub_axi_awtrace     (fub_awtrace),
.fub_axi_awmpam      (fub_awmpam),
.fub_axi_awmecid     (fub_awmecid),
.fub_axi_awunique    (fub_awunique),
.fub_axi_awtagop     (fub_awtagop),
.fub_axi_awtag       (fub_awtag),

// W channel
.fub_axi_wdata       (fub_wdata),
.fub_axi_wstrb       (fub_wstrb),
.fub_axi_wlast       (fub_wlast),
.fub_axi_wuser       (fub_wuser),
.fub_axi_wvalid      (fub_wvalid),
.fub_axi_wready      (fub_wready),

// AXI5 W extensions
.fub_axi_wpoison     (fub_wpoison),

```

```

.fub_axi_wtag      (fub_wtag),
.fub_axi_wtagupdate (fub_wtagupdate),

// B channel
.fub_axi_bid      (fub_bid),
.fub_axi_bresp    (fub_bresp),
.fub_axi_buser    (fub_buser),
.fub_axi_bvalid   (fub_bvalid),
.fub_axi_bready   (fub_bready),

// AXI5 B extensions
.fub_axi_btrace   (fub_btrace),
.fub_axi_btag     (fub_btag),
.fub_axi_btagmatch (fub_btagmatch),

// Master interface (output side)
.m_axi_awid      (m_axi_awid),
.m_axi_awaddr    (m_axi_awaddr),
//... (connect all master signals similarly)

// Status
.busy            (master_wr_busy)
);

```

4.7 Design Notes

4.7.1 Atomic Operation Support

When `ENABLE_ATOMIC=1`: - **AWATOP[5:4]** selects atomic type (Swap/Compare/Fetch-op) - **AWATOP[3:0]** selects operation for Fetch-op - Write data provides operand value - Response may include atomic result (implementation-dependent)

Common atomic operations: - **Atomic Swap**: Exchange memory value with register - **Compare and Swap**: Update memory if value matches - **Fetch and Add**: Read-modify-write atomically

4.7.2 Memory Tagging Extension (MTE)

When `ENABLE_MTE=1`: - **AWTAGOP**: Specifies tag operation for address tags - **AWTAG**: Provides address-level memory tags - **WTAG**: Provides data-level memory tags (1 tag per 16 bytes) - **WTAGUPDATE**: Mask for selective tag updates - **BTAG/BTAGMATCH**: Response includes tag information

4.7.3 Poison Support

When `ENABLE_POISON=1`: - **WPOISON**: Master flags data as poisoned - Use cases: Error injection, cache pollution testing, security

4.7.4 Feature Enable Strategy

Disable unused features to reduce area:

```
.ENABLE_ATOMIC    (0), // No atomic operations
.ENABLE_NSaid     (0), // No security domains
.ENABLE_TRACE     (0), // No trace capability
.ENABLE_MPAM      (0), // No memory partitioning
.ENABLE_MECID     (0), // No encryption
.ENABLE_UNIQUE    (0), // No unique access
.ENABLE_MTE       (0), // No memory tagging
.ENABLE_POISON    (0)  // No poison indication
```

4.8 Related Documentation

- [AXI5 Master Read](#) - Master read interface
 - [AXI5 Slave Write](#) - Slave write interface
 - [AXI5 Master Write CG](#) - Clock-gated variant
 - [AXI5 Master Write Monitor](#) - With integrated monitoring
 - [AXI4 Master Write](#) - AXI4 version for comparison
-

4.9 Navigation

- [← Back to AXI5 Index](#)
- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

5 AXI5 Slave Read with Clock Gating

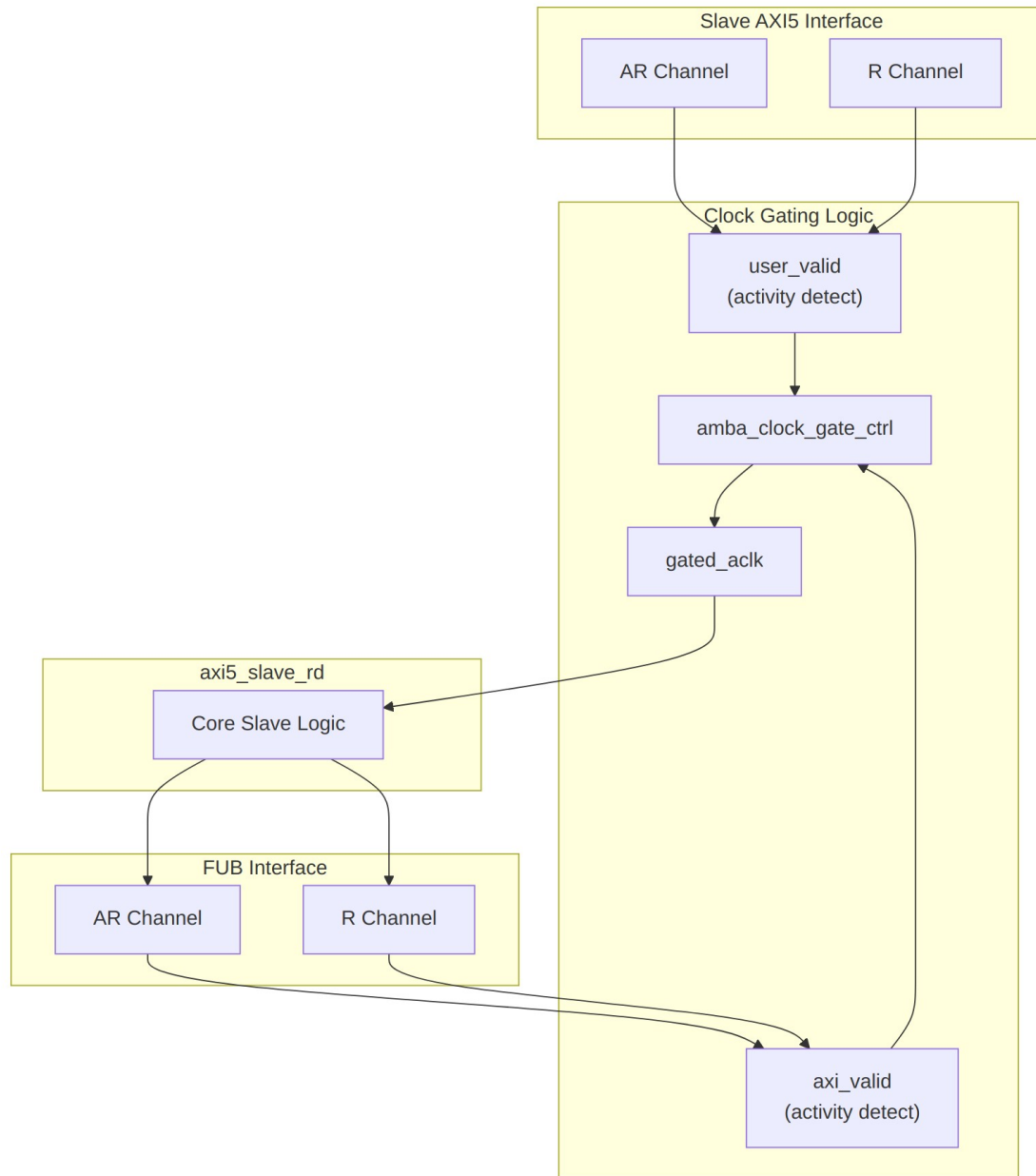
Module: `axi5_slave_rd_cg.sv` **Location:** `rtl/amba/axi5/` **Status:** Production Ready

5.1 Overview

The AXI5 Slave Read with Clock Gating module wraps the standard `axi5_slave_rd` module with integrated clock gating for power optimization. It automatically gates the internal clock when the module is idle.

5.1.1 Key Features

- Full AMBA AXI5 slave read protocol compliance
 - **Integrated clock gating** for dynamic power reduction
 - Configurable idle count before gating
 - All AXI5 extensions supported (NSAID, TRACE, MPAM, MECID, UNIQUE, CHUNKING, MTE, POISON)
 - Transparent gating - no protocol changes
 - Gating status outputs for system monitoring
 - SKID buffering for AR and R channels
-



Module Architecture

5.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_A R	int	2	AR channel SKID buffer

Parameter	Type	Default	Description
			depth
SKID_DEPTH_R	int	4	R channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width
AXI_MECID_WIDTH	int	16	Memory encryption context width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
AXI_CHUNKNUM_WIDTH	int	4	Chunk number width
ENABLE_NSAI	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPA	bit	1	Enable memory

Parameter	Type	Default	Description
ENABLE_MECID	bit	1	partitioning Enable memory encryption
ENABLE_UNIQUE	bit	1	Enable unique ID indicator
ENABLE_CHUNKING	bit	1	Enable data chunking
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
CG_IDLE_COUNTER_WIDTH	int	4	Clock gating idle counter width

5.3 Ports

5.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock (ungated)
aresetn	1	Input	AXI active-low reset

5.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Enable clock gating
cfg_cg_idle_count	CG_IDLE_COUNTER_WIDTH	Input	Idle cycles before gating

5.3.3 Slave AXI5 Interface

Same as axi5_slave_rd - see [AXI5 Slave Read](#) for complete port list.

5.3.4 FUB Interface

Same as axi5_slave_rd - see [AXI5 Slave Read](#) for complete port list.

5.3.5 Clock Gating Status

Port	Width	Direction	Description
cg_gating	1	Output	Clock is currently gated
cg_idle	1	Output	Module is idle

5.4 Functionality

5.4.1 Clock Gating Behavior

Activity Detection: - **user_valid:** Asserted when slave interface has activity (arvalid, rready, or internal busy) - **axi_valid:** Asserted when FUB interface has activity (arvalid, rvalid)

Gating State Machine:

```
stateDiagram-v2
    [*] --> ACTIVE

    ACTIVE --> COUNTING : !user_valid && !axi_valid
    COUNTING --> ACTIVE : user_valid || axi_valid
    COUNTING --> GATED : count == cfg_cg_idle_count
    GATED --> ACTIVE : user_valid || axi_valid

    state ACTIVE {
        note right of ACTIVE : Clock enabled<br/>cg_gating = 0
    }
    state COUNTING {
        note right of COUNTING : Idle countdown<br/>cg_gating = 0
    }
    state GATED {
        note right of GATED : Clock gated<br/>cg_gating = 1
    }
```

Key Points: - Clock gating disabled when cfg_cg_enable = 0 - Ready signals forced to 0 when gated (prevents new transactions) - Gating only occurs after configured idle period - Any activity immediately ungates the clock

5.5 Clock Gating Configuration

5.5.1 Idle Count Selection

cfg_cg_idle_count	Idle Cycles	Use Case
0	1	Aggressive power saving, frequent gating
1-3	2-8	Balanced, reduces gate churn
4-7	16-128	Conservative, for bursty traffic
8+	256+	Minimal gating, continuous operation

Recommendations: - **Streaming workloads:** Higher count (4-8) to avoid frequent gate/ungate - **Sporadic transactions:** Lower count (0-2) for maximum power savings - **Real-time systems:** Disable gating or use high count to ensure deterministic latency

5.6 Timing Diagrams

5.6.1 Clock Gating Sequence

TODO: Wavedrom showing:

- aclk (ungated)
 - gated_aclk
 - user_valid
 - axi_valid
 - cg_idle
 - cg_gating
 - idle count progression
-

5.7 Usage Example

```
axi5_slave_rd_cg #(  
    .AXI_ID_WIDTH      (8),  
    .AXI_ADDR_WIDTH    (32),
```

```

        .AXI_DATA_WIDTH      (64),
        .SKID_DEPTH_AR      (2),
        .SKID_DEPTH_R       (4),
        .CG_IDLE_COUNT_WIDTH(4),
        .ENABLE_NSAID       (1),
        .ENABLE_TRACE       (1),
        .ENABLE_MPAM        (1),
        .ENABLE_MECID       (1),
        .ENABLE_UNIQUE      (1),
        .ENABLE_CHUNKING    (1),
        .ENABLE_MTE         (1),
        .ENABLE_POISON      (1)
    ) u_axi5_slave_rd_cg (
        .aclk                (axi_clk),
        .aresetn             (axi_rst_n),

        // Clock gating config
        .cfg_cg_enable       (1'b1),           // Enable gating
        .cfg_cg_idle_count   (4'd3),           // Gate after 8 idle cycles

        // Slave interface (from external master)
        .s_axi_arid          (s_axi_arid),
        .s_axi_araddr        (s_axi_araddr),
        // ... (connect all slave AR/R signals)

        // FUB interface (to backend)
        .fub_axi_arid        (mem_arid),
        .fub_axi_araddr      (mem_araddr),
        // ... (connect to memory controller)

        // Clock gating status
        .cg_gating           (slave_rd_gating),
        .cg_idle             (slave_rd_idle)
    );

    // Optional: Monitor power savings
    always @(posedge axi_clk) begin
        if (slave_rd_gating)
            $display("AXI5 Slave RD clock gated - saving power");
    end

```

5.8 Design Notes

5.8.1 Power Savings Estimation

Assuming 60% module activity: - **Without gating:** 100% dynamic power - **With gating (idle_count=2):** ~50% dynamic power (40% gated, accounting for transitions) - **With gating (idle_count=0):** ~60% dynamic power (more aggressive but more transitions)

5.8.2 Clock Gating Overhead

- **Area:** ~2-5% increase (clock gate cells, idle counter)
- **Timing:** Clock gating adds minimal delay (typically <50ps)
- **Power:** Overhead from gate control logic usually <1% of savings

5.8.3 When to Use Clock Gating

Good candidates: - Low-duty-cycle interfaces (sporadic transactions) - Systems with strict power budgets - Battery-operated devices

Avoid when: - Interface is continuously active - Gate/ungate transitions exceed power savings - Deterministic latency required (use high idle count instead)

5.8.4 Integration with Power Management

```
// System power controller
assign axi_power_save_mode = slave_rd_gating &&
                             slave_wr_gating &&
                             master_rd_gating;

// Optional: Track gating efficiency
always @(posedge axi_clk or negedge aresetn) begin
    if (!aresetn) begin
        gating_cycles <= 0;
        total_cycles <= 0;
    end else begin
        total_cycles <= total_cycles + 1;
        if (slave_rd_gating)
            gating_cycles <= gating_cycles + 1;
    end
end

assign gating_efficiency = (gating_cycles * 100) / total_cycles;
```

5.9 Related Documentation

- [AXI5 Slave Read](#) - Non-clock-gated version
 - [AXI5 Slave Write CG](#) - Clock-gated write variant
 - [AXI5 Slave Read Monitor CG](#) - With monitoring
 - [AMBA Clock Gate Control](#) - Clock gating controller
-

5.10 Navigation

- [← Back to AXI5 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

6 AXI5 Slave Read

Module: `axi5_slave_rd.sv` **Location:** `rtl/amba/axi5/` **Status:** Production Ready

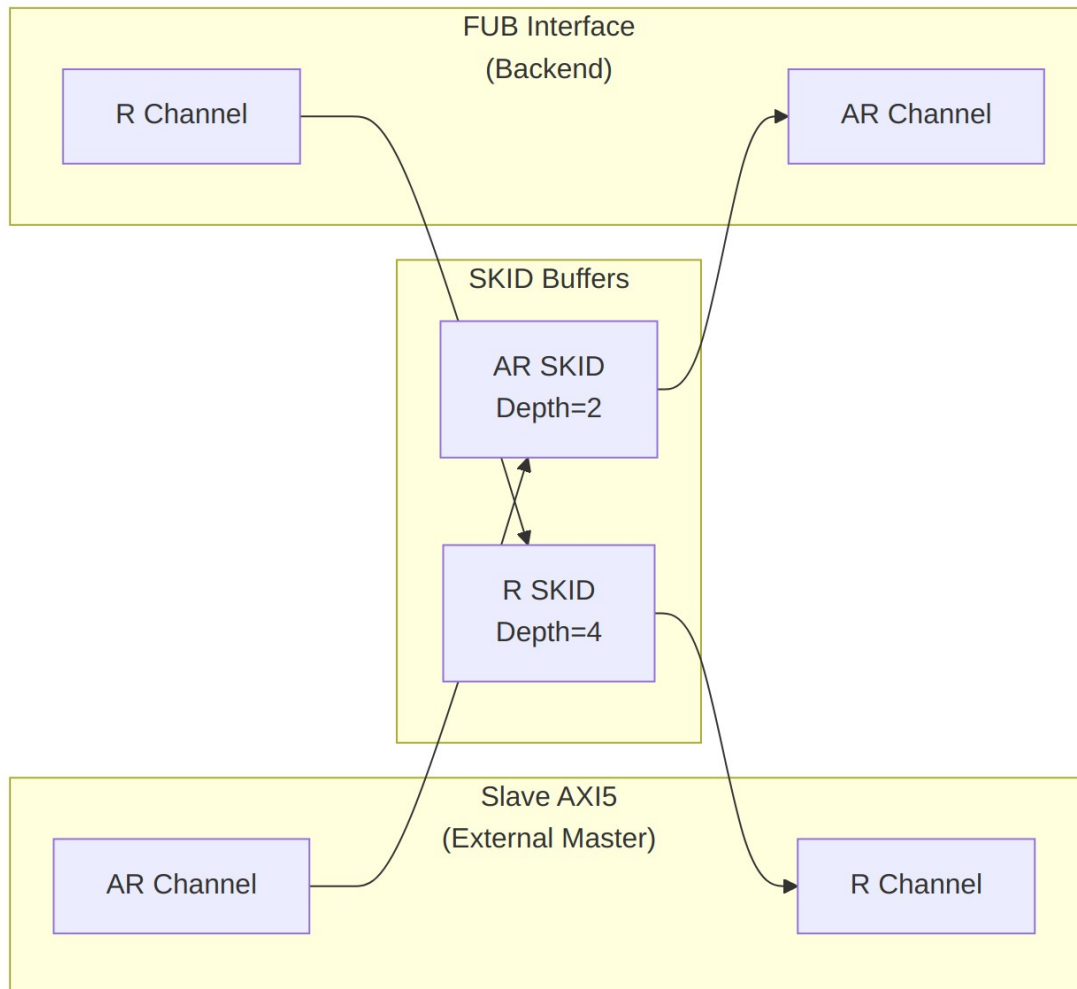
6.1 Overview

The AXI5 Slave Read module implements a complete AMBA AXI5 slave read interface with full AXI5 protocol support. It receives read requests from external masters and forwards them to backend memory/logic (FUB interface).

6.1.1 Key Features

- Full AMBA AXI5 slave protocol compliance
- **ARNSAID:** Non-secure access identifier reception
- **ARTRACE:** Trace signal support
- **ARMPAM:** Memory Partitioning and Monitoring reception
- **ARMECID:** Memory Encryption Context ID reception
- **ARUNIQUE:** Unique ID indicator reception
- **ARCHUNKEN:** Chunking enable reception
- **ARTAGOP:** Memory tag operation reception (MTE)
- **RTRACE/RPOISON:** Response extensions
- **RCHUNKV/RCHUNKNUM/RCHUNKSTRB:** Chunking support
- **RTAG/RTAGMATCH:** Memory tag responses (MTE)
- SKID buffering for AR and R channels

- Busy signal for power management



Module Architecture

6.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_A R	int	2	AR channel SKID buffer depth
SKID_DEPTH_R	int	4	R channel SKID buffer depth

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width
AXI_MECID_WIDTH	int	16	Memory encryption context width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
AXI_CHUNKNUMBER_WIDTH	int	4	Chunk number width
ENABLE_NSAI	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPA	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption

Parameter	Type	Default	Description
ENABLE_UNIQ UE	bit	1	Enable unique ID indicator
ENABLE_CHUN KING	bit	1	Enable data chunking
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POIS ON	bit	1	Enable poison indicator

6.3 Ports

6.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

6.3.2 Slave AXI5 Interface (External Master Input)

6.3.2.1 AR Channel (Input)

Port	Width	Direction	Description
s_axi_arid	IW	Input	Read address ID
s_axi_arad dr	AW	Input	Read address
s_axi_arlen	8	Input	Burst length
s_axi_arsiz e	3	Input	Burst size
s_axi_arbu rst	2	Input	Burst type
s_axi_arloc k	1	Input	Lock type
s_axi_arcac	4	Input	Cache attributes

Port	Width	Direction	Description
he			
s_axi_arprot	3	Input	Protection attributes
s_axi_arqos	4	Input	Quality of Service
s_axi_aruser	UW	Input	User-defined signal
s_axi_arvalid	1	Input	Read address valid
s_axi_arready	1	Output	Read address ready
s_axi_arnsid	AXI_NSaid_WIDTH	Input	Non-secure access ID (AXI5)
s_axi_artrace	1	Input	Trace signal (AXI5)
s_axi_armpam	AXI_MPAM_WIDTH	Input	Memory partitioning (AXI5)
s_axi_armecid	AXI_MECID_WIDTH	Input	Memory encryption context (AXI5)
s_axi_arunique	1	Input	Unique ID indicator (AXI5)
s_axi_archunken	1	Input	Chunking enable (AXI5)
s_axi_artagop	AXI_TAGOP_WIDTH	Input	Tag operation (AXI5 MTE)

6.3.2.2 R Channel (Output)

Port	Width	Direction	Description
s_axi_rid	IW	Output	Read data ID
s_axi_rdata	DW	Output	Read data
s_axi_rresp	2	Output	Read response
s_axi_rlast	1	Output	Last transfer in burst

Port	Width	Direction	Description
s_axi_ruser	UW	Output	User-defined signal
s_axi_rvalid	1	Output	Read data valid
s_axi_rready	1	Input	Read data ready
s_axi_rtrace	1	Output	Trace signal (AXI5)
s_axi_rpoison	1	Output	Data poison indicator (AXI5)
s_axi_rchunkv	1	Output	Chunk valid (AXI5)
s_axi_rchunknum	AXI_CHUNKNUM_WIDTH	Output	Chunk number (AXI5)
s_axi_rchunkstrob	CHUNK_STRB_WIDTH	Output	Chunk strobe (AXI5)
s_axi_rtag	TW	Output	Memory tags (AXI5 MTE)
s_axi_rtagmatch	1	Output	Tag match response (AXI5 MTE)

6.3.3 FUB Interface (Backend Memory/Logic Output)

Same signals as slave interface but with fub_axi_* prefix and reversed directions for AR (output) and R (input).

6.3.4 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy indicator

6.4 Functionality

This slave module acts as an adapter between: - **External AXI5 master** (s_axi_* interface) - **Backend memory or functional unit block** (fub_axi_* interface)

Signal Flow: 1. AR channel: External master → SKID → Backend 2. R channel: Backend → SKID → External master

SKID Buffering: Provides elasticity and breaks combinational paths for timing closure.

6.5 Timing Diagrams

6.5.1 Basic Read Transaction

TOD0: Wavedrom showing AR request from external master flowing through to backend FUB interface

6.6 Usage Example

```
axi5_slave_rd #(
    .AXI_ID_WIDTH      (8),
    .AXI_ADDR_WIDTH    (32),
    .AXI_DATA_WIDTH    (64),
    .SKID_DEPTH_AR     (2),
    .SKID_DEPTH_R      (4),
    .ENABLE_NSAID      (1),
    .ENABLE_TRACE      (1),
    .ENABLE_MPAM       (1),
    .ENABLE_MECID      (1),
    .ENABLE_UNIQUE     (1),
    .ENABLE_CHUNKING   (1),
    .ENABLE_MTE        (1),
    .ENABLE_POISON     (1)
) u_axi5_slave_rd (
    .aclk               (axi_clk),
    .aresetn            (axi_rst_n),

    // Slave interface (from external master)
    .s_axi_arid         (s_axi_arid),
    .s_axi_araddr       (s_axi_araddr),
    .s_axi_arlen        (s_axi_arlen),
    // ... (connect all slave AR signals)
    .s_axi_arnsaid      (s_axi_arnsaid),
    .s_axi_artrace      (s_axi_artrace),
```

```
// ... (connect AXI5 extensions)

.s_axi_rid      (s_axi_rid),
.s_axi_rdata    (s_axi_rdata),
// ... (connect all slave R signals)

// FUB interface (to backend)
.fub_axi_arid    (mem_arid),
.fub_axi_araddr  (mem_araddr),
// ... (connect to memory controller)

.busy            (slave_rd_busy)
);
```

6.7 Design Notes

- Use when you need AXI5 slave functionality with SKID buffering
 - Backend FUB interface can connect to memory controller, SRAM, or functional units
 - SKID buffers improve timing by breaking long combinational paths
 - Feature enables allow area optimization by disabling unused AXI5 extensions
-

6.8 Related Documentation

- [AXI5 Slave Write](#) - Slave write interface
 - [AXI5 Master Read](#) - Master read interface
 - [AXI5 Slave Read CG](#) - Clock-gated variant
 - [AXI5 Slave Read Monitor](#) - With integrated monitoring
-

6.9 Navigation

- [← Back to AXI5 Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

7 AXI5 Slave Write with Clock Gating

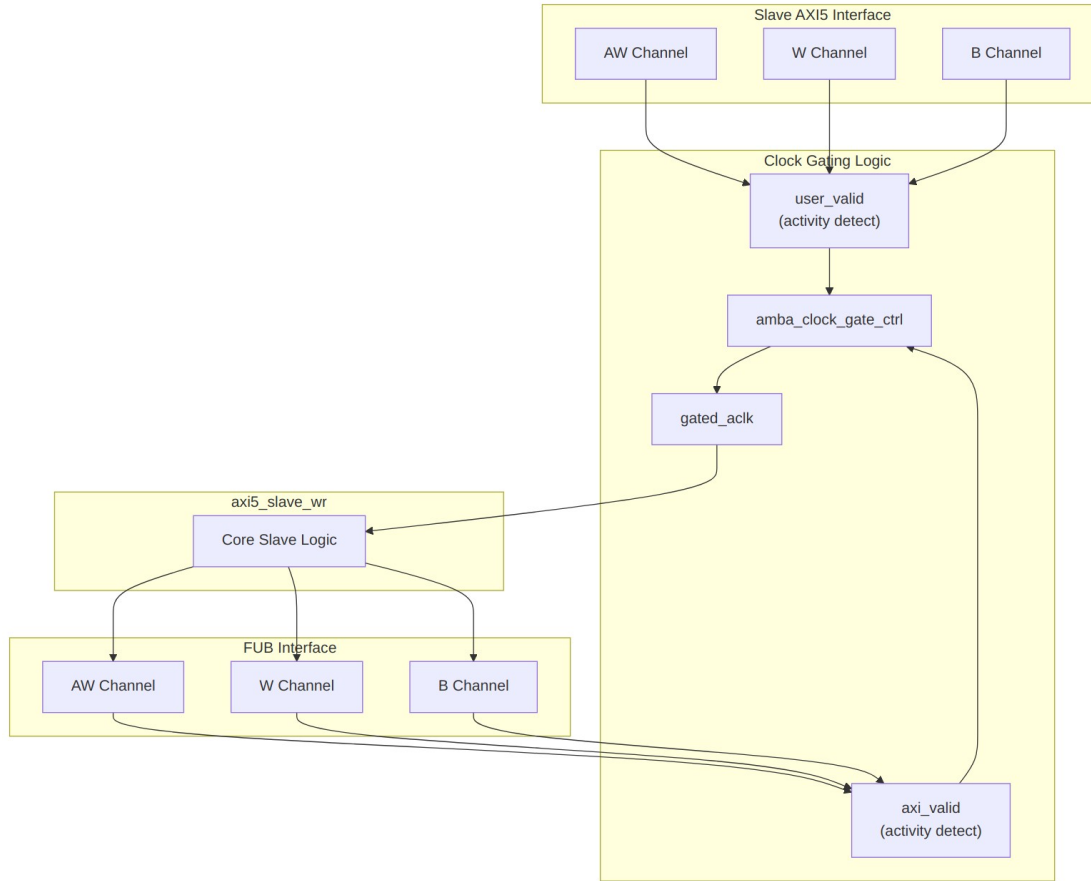
Module: axi5_slave_wr_cg.sv **Location:** rtl/amba/axi5/ **Status:** Production Ready

7.1 Overview

The AXI5 Slave Write with Clock Gating module wraps the standard axi5_slave_wr module with integrated clock gating for power optimization. It automatically gates the internal clock when the module is idle.

7.1.1 Key Features

- Full AMBA AXI5 slave write protocol compliance
 - **Integrated clock gating** for dynamic power reduction
 - Configurable idle count before gating
 - All AXI5 extensions supported (ATOMIC, NSAID, TRACE, MPAM, MECID, UNIQUE, MTE, POISON)
 - Transparent gating - no protocol changes
 - Gating status outputs for system monitoring
 - SKID buffering for AW, W, and B channels
-



Module Architecture

7.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_A W	int	2	AW channel SKID buffer depth
SKID_DEPTH_ W	int	4	W channel SKID buffer depth
SKID_DEPTH_B	int	2	B channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width

Parameter	Type	Default	Description
AXI_ADDR_WI DTH	int	32	Address bus width
AXI_DATA_WI DTH	int	32	Data bus width
AXI_USER_WID TH	int	1	User signal width
AXI_ATOP_WI DTH	int	6	Atomic operation width
AXI_NSAID_WI DTH	int	4	Non-secure access ID width
AXI_MPAM_WI DTH	int	11	MPAM width
AXI_MECID_WI DTH	int	16	Memory encryption context width
AXI_TAG_WID TH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_W IDTH	int	2	Tag operation width
ENABLE_ATO MIC	bit	1	Enable atomic operations
ENABLE_NSAI D	bit	1	Enable non-secure access ID
ENABLE_TRAC E	bit	1	Enable trace signals
ENABLE_MPA M	bit	1	Enable memory partitioning
ENABLE_MECI D	bit	1	Enable memory

Parameter	Type	Default	Description
ENABLE_UNIQ	bit	1	Enable unique ID indicator
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
CG_IDLE_COUNTER_WIDTH	int	4	Clock gating idle counter width

7.3 Ports

7.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock (ungated)
aresetn	1	Input	AXI active-low reset

7.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Enable clock gating
cfg_cg_idle_count	CG_IDLE_COUNTER_WIDTH	Input	Idle cycles before gating

7.3.3 Slave AXI5 Interface

Same as axi5_slave_wr - see [AXI5 Slave Write](#) for complete port list.

7.3.4 FUB Interface

Same as axi5_slave_wr - see [AXI5 Slave Write](#) for complete port list.

7.3.5 Clock Gating Status

Port	Width	Direction	Description
cg_gating	1	Output	Clock is currently gated
cg_idle	1	Output	Module is idle

7.4 Functionality

7.4.1 Clock Gating Behavior

Activity Detection: - **user_valid:** Asserted when slave interface has activity (awvalid, wvalid, bready, or internal busy) - **axi_valid:** Asserted when FUB interface has activity (awvalid, wvalid, bvalid)

Gating State Machine:

```
stateDiagram-v2
    [*] --> ACTIVE

    ACTIVE --> COUNTING : !user_valid && !axi_valid
    COUNTING --> ACTIVE : user_valid || axi_valid
    COUNTING --> GATED : count == cfg_cg_idle_count
    GATED --> ACTIVE : user_valid || axi_valid

    state ACTIVE {
        note right of ACTIVE : Clock enabled<br/>cg_gating = 0
    }
    state COUNTING {
        note right of COUNTING : Idle countdown<br/>cg_gating = 0
    }
    state GATED {
        note right of GATED : Clock gated<br/>cg_gating = 1
    }
```

Key Points: - Clock gating disabled when `cfg_cg_enable = 0` - Ready signals (awready, wready) forced to 0 when gated (prevents new transactions) - bready forced to 0 when gated (prevents accepting responses) - Gating only occurs after configured idle period - Any activity immediately ungates the clock

7.5 Clock Gating Configuration

7.5.1 Idle Count Selection

cfg_cg_idle_count	Idle Cycles	Use Case
0	1	Aggressive power saving, frequent gating
1-3	2-8	Balanced, reduces gate churn
4-7	16-128	Conservative, for bursty traffic
8+	256+	Minimal gating, continuous operation

Recommendations: - **Burst writes:** Higher count (4-8) to avoid gating mid-burst - **Sporadic writes:** Lower count (0-2) for maximum power savings - **Real-time systems:** Disable gating or use high count to ensure deterministic latency

7.6 Timing Diagrams

7.6.1 Clock Gating During Write Burst

TODO: Wavedrom showing:

- aclk (ungated)
 - gated_aclk
 - awvalid, wvalid, bvalid
 - user_valid, axi_valid
 - cg_idle, cg_gating
 - Burst write transaction
 - Gating after burst completion
-

7.7 Usage Example

```
axi5_slave_wr_cg #(
    .AXI_ID_WIDTH      (8),
    .AXI_ADDR_WIDTH    (32),
    .AXI_DATA_WIDTH    (64),
    .SKID_DEPTH_AW     (2),
```

```

        .SKID_DEPTH_W          (4),
        .SKID_DEPTH_B          (2),
        .CG_IDLE_COUNT_WIDTH  (4),
        .ENABLE_ATOMIC         (1),
        .ENABLE_NSAID          (1),
        .ENABLE_TRACE          (1),
        .ENABLE_MPAM           (1),
        .ENABLE_MECID          (1),
        .ENABLE_UNIQUE         (1),
        .ENABLE_MTE            (1),
        .ENABLE_POISON         (1)
    ) u_axi5_slave_wr_cg (
        .aclk                    (axi_clk),
        .aresetn                 (axi_rst_n),

        // Clock gating config
        .cfg_cg_enable           (1'b1),                // Enable gating
        .cfg_cg_idle_count      (4'd3),                // Gate after 8 idle cycles

        // Slave interface (from external master)
        .s_axi_awid              (s_axi_awid),
        .s_axi_awaddr            (s_axi_awaddr),
        // ... (connect all slave AW/W/B signals)

        // FUB interface (to backend)
        .fub_axi_awid            (mem_awid),
        .fub_axi_awaddr          (mem_awaddr),
        // ... (connect to memory controller)

        // Clock gating status
        .cg_gating               (slave_wr_gating),
        .cg_idle                 (slave_wr_idle)
    );

    // Optional: Monitor power savings
    always @(posedge axi_clk) begin
        if (slave_wr_gating)
            $display("AXI5 Slave WR clock gated - saving power");
    end

    // Power management integration
    assign system_low_power = slave_wr_gating &&
        slave_rd_gating;

```

7.8 Design Notes

7.8.1 Power Savings Estimation

Assuming 60% module activity: - **Without gating:** 100% dynamic power - **With gating (idle_count=2):** ~50% dynamic power (40% gated, accounting for transitions) - **With gating (idle_count=0):** ~60% dynamic power (more aggressive but more transitions)

7.8.2 Write-Specific Considerations

Burst Handling: - W channel SKID depth (4) larger than AW/B (2) to accommodate burst data - Gating should not occur mid-burst (use adequate idle_count) - Burst latency unaffected if gating disabled during active bursts

Atomic Operations: - Atomic writes may have longer latencies - Consider higher idle_count when ENABLE_ATOMIC=1 - Gating safe between atomic transactions

7.8.3 Clock Gating Overhead

- **Area:** ~2-5% increase (clock gate cells, idle counter)
- **Timing:** Clock gating adds minimal delay (typically <50ps)
- **Power:** Overhead from gate control logic usually <1% of savings

7.8.4 When to Use Clock Gating

Good candidates: - Low-duty-cycle interfaces (sporadic transactions) - Systems with strict power budgets - Battery-operated devices - Write-infrequent applications (logging, configuration)

Avoid when: - Interface is continuously active (streaming writes) - Gate/ungate transitions exceed power savings - Deterministic latency required (use high idle count instead) - Write bursts dominate (minimal idle periods)

7.9 Related Documentation

- [AXI5 Slave Write](#) - Non-clock-gated version
 - [AXI5 Slave Read CG](#) - Clock-gated read variant
 - [AXI5 Slave Write Monitor CG](#) - With monitoring
 - [AMBA Clock Gate Control](#) - Clock gating controller
-

7.10 Navigation

- [← Back to AXI5 Index](#)

- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

8 AXI5 Slave Write

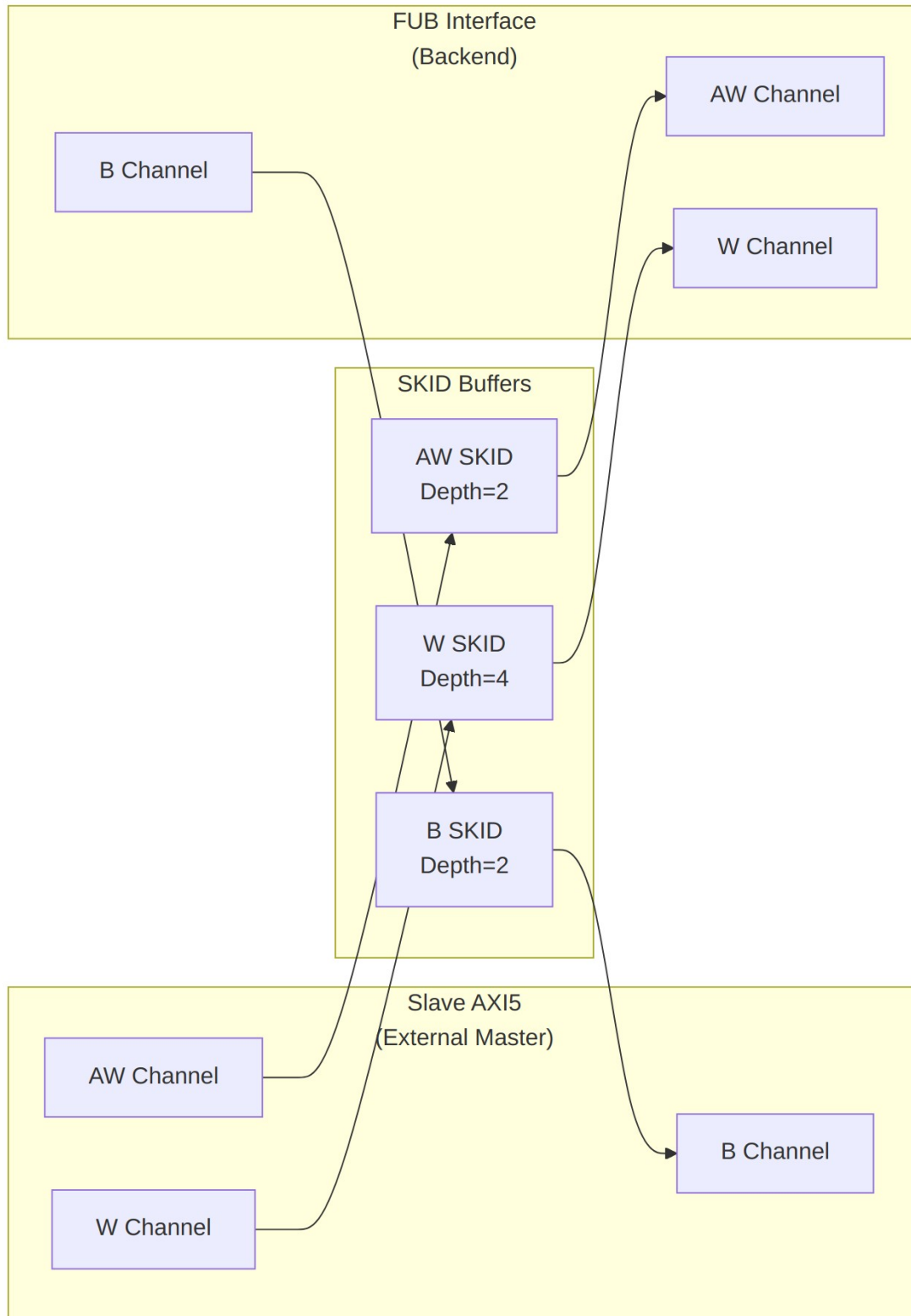
Module: `axi5_slave_wr.sv` **Location:** `rtl/amba/axi5/` **Status:** Production Ready

8.1 Overview

The AXI5 Slave Write module implements a complete AMBA AXI5 slave write interface with full AXI5 protocol support. It receives write requests from external masters and forwards them to backend memory/logic (FUB interface).

8.1.1 Key Features

- Full AMBA AXI5 slave protocol compliance
 - **AWATOP:** Atomic transaction operation type
 - **AWNSAID:** Non-secure access identifier reception
 - **AWTRACE:** Trace signal support
 - **AWMPAM:** Memory Partitioning and Monitoring reception
 - **AWMECID:** Memory Encryption Context ID reception
 - **AWUNIQUE:** Unique ID indicator reception
 - **AWTAGOP/AWTAG:** Memory tag operation reception (MTE)
 - **WPOISON:** Write data poison indicator
 - **WTAG/WTAGUPDATE:** Write data memory tags (MTE)
 - **BTRACE/BTAG/BTAGMATCH:** Response extensions
 - SKID buffering for AW, W, and B channels
 - Busy signal for power management
 - Deprecated AWREGION removed (not recommended for new designs)
-



Module Architecture

8.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel SKID buffer depth
SKID_DEPTH_W	int	4	W channel SKID buffer depth
SKID_DEPTH_B	int	2	B channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_ATOP_WIDTH	int	6	Atomic operation width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width
AXI_MECID_WIDTH	int	16	Memory encryption context width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_W	int	2	Tag operation

Parameter	Type	Default	Description
IDTH			width
ENABLE_ATO MIC	bit	1	Enable atomic operations
ENABLE_NSAI D	bit	1	Enable non-secure access ID
ENABLE_TRAC E	bit	1	Enable trace signals
ENABLE_MPA M	bit	1	Enable memory partitioning
ENABLE_MECI D	bit	1	Enable memory encryption
ENABLE_UNIQ UE	bit	1	Enable unique ID indicator
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POIS ON	bit	1	Enable poison indicator

8.3 Ports

8.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

8.3.2 Slave AXI5 Interface (External Master Input)

8.3.2.1 AW Channel (Input)

Port	Width	Direction	Description
s_axi_awid	IW	Input	Write address ID
s_axi_awa ddr	AW	Input	Write address
s_axi_awle n	8	Input	Burst length
s_axi_aws ize	3	Input	Burst size
s_axi_awb urst	2	Input	Burst type
s_axi_awlo ck	1	Input	Lock type
s_axi_awca che	4	Input	Cache attributes
s_axi_awpr ot	3	Input	Protection attributes
s_axi_awq os	4	Input	Quality of Service
s_axi_awus er	UW	Input	User-defined signal
s_axi_awva lid	1	Input	Write address valid
s_axi_awre ady	1	Output	Write address ready
s_axi_awa t op	AXI_ATOP_W IDTH	Input	Atomic operation (AXI5)
s_axi_awns aid	AXI_NSAID_ WIDTH	Input	Non-secure access ID (AXI5)
s_axi_awtr ace	1	Input	Trace signal (AXI5)
s_axi_awm pam	AXI_MPAM_ WIDTH	Input	Memory partitioning (AXI5)

Port	Width	Direction	Description
s_axi_awm ecid	AXI_MECID_ WIDTH	Input	Memory encryption context (AXI5)
s_axi_awu nique	1	Input	Unique ID indicator (AXI5)
s_axi_awta gop	AXI_TAGOP_ WIDTH	Input	Tag operation (AXI5 MTE)
s_axi_awta g	TW	Input	Memory tags (AXI5 MTE)

8.3.2.2 W Channel (Input)

Port	Width	Direction	Description
s_axi_wdata	DW	Input	Write data
s_axi_wstrb	SW	Input	Write strobes
s_axi_wlast	1	Input	Last transfer in burst
s_axi_wuser	UW	Input	User-defined signal
s_axi_wvalid	1	Input	Write data valid
s_axi_wready	1	Output	Write data ready
s_axi_wpoison	1	Input	Data poison indicator (AXI5)
s_axi_wtag	TW	Input	Write data memory tags (AXI5 MTE)
s_axi_wtagupd ate	NUM_TAGS	Input	Tag update mask (AXI5 MTE)

8.3.2.3 B Channel (Output)

Port	Width	Direction	Description
s_axi_bid	IW	Output	Write response

Port	Width	Direction	Description
			ID
s_axi_bresp	2	Output	Write response
s_axi_buser	UW	Output	User-defined signal
s_axi_bvalid	1	Output	Write response valid
s_axi_bready	1	Input	Write response ready
s_axi_btrace	1	Output	Trace signal (AXI5)
s_axi_btag	TW	Output	Memory tags (AXI5 MTE)
s_axi_btagmatch	1	Output	Tag match response (AXI5 MTE)

8.3.3 FUB Interface (Backend Memory/Logic Output)

Same signals as slave interface but with `fub_axi_*` prefix and reversed directions for AW/W (output) and B (input).

8.3.4 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy indicator

8.4 Functionality

This slave module acts as an adapter between: - **External AXI5 master** (`s_axi_*` interface) - **Backend memory or functional unit block** (`fub_axi_*` interface)

Signal Flow: 1. AW channel: External master → SKID → Backend 2. W channel: External master → SKID → Backend 3. B channel: Backend → SKID → External master

SKID Buffering: Provides elasticity and breaks combinational paths for timing closure.

Busy Signal: Asserted when any SKID buffer has data or handshakes are pending.

8.5 Timing Diagrams

8.5.1 Basic Write Transaction

TOD0: Wavedrom showing AW/W request from external master flowing through to backend FUB interface

8.5.2 Write with Poison Indicator

TOD0: Wavedrom showing write transaction with WPOISON asserted

8.6 Usage Example

```
axi5_slave_wr #(
    .AXI_ID_WIDTH          (8),
    .AXI_ADDR_WIDTH        (32),
    .AXI_DATA_WIDTH        (64),
    .SKID_DEPTH_AW         (2),
    .SKID_DEPTH_W          (4),
    .SKID_DEPTH_B          (2),
    .ENABLE_ATOMIC         (1),
    .ENABLE_NSaid          (1),
    .ENABLE_TRACE          (1),
    .ENABLE_MPAM           (1),
    .ENABLE_MECID          (1),
    .ENABLE_UNIQUE         (1),
    .ENABLE_MTE            (1),
    .ENABLE_POISON         (1)
) u_axi5_slave_wr (
    .aclk                  (axi_clk),
    .aresetn               (axi_rst_n),

    // Slave interface (from external master)
    .s_axi_awid            (s_axi_awid),
    .s_axi_awaddr          (s_axi_awaddr),
    .s_axi_awlen           (s_axi_awlen),
    // ... (connect all slave AW signals)
    .s_axi_awatop          (s_axi_awatop),
    .s_axi_awnsaid         (s_axi_awnsaid),
    .s_axi_awtrace         (s_axi_awtrace),
    // ... (connect AXI5 extensions)

    .s_axi_wdata           (s_axi_wdata),
    .s_axi_wstrb           (s_axi_wstrb),
```

```

// ... (connect all slave W signals)

.s_axi_bid          (s_axi_bid),
.s_axi_bresp        (s_axi_bresp),
// ... (connect all slave B signals)

// FUB interface (to backend)
.fub_axi_awid        (mem_awid),
.fub_axi_awaddr      (mem_awaddr),
// ... (connect to memory controller)

.busy                (slave_wr_busy)
);

```

8.7 Design Notes

- Use when you need AXI5 slave write functionality with SKID buffering
 - Backend FUB interface can connect to memory controller, SRAM, or functional units
 - SKID buffers improve timing by breaking long combinational paths
 - W channel SKID depth typically larger than AW/B due to burst data
 - Feature enables allow area optimization by disabling unused AXI5 extensions
 - AWREGION deprecated in AXI5 (not recommended for new designs)
-

8.8 Related Documentation

- [AXI5 Slave Read](#) - Slave read interface
 - [AXI5 Master Write](#) - Master write interface
 - [AXI5 Slave Write CG](#) - Clock-gated variant
 - [AXI5 Slave Write Monitor](#) - With integrated monitoring
-

8.9 Navigation

- [← Back to AXI5 Index](#)
- [← Back to RTLamba Index](#)
- [← Back to Main Documentation Index](#)

9 AXI5 (Advanced eXtensible Interface - AMBA 5) Modules

Location: rtl/amba/axi5/ **Test Location:** val/amba/ **Status:** Production Ready

9.1 Overview

The AXI5 subsystem provides a comprehensive implementation of the ARM AMBA 5 AXI (Advanced eXtensible Interface) protocol, including read/write masters and slaves, integrated monitors, clock-gated variants, and combined master+monitor modules for efficient system integration.

AXI5 extends AXI4 with significant enhancements for modern high-performance SoC designs, including atomic operations, quality of service improvements, memory tagging, and enhanced coherency support.

9.2 AMBA4 vs AMBA5 Comparison

Feature	AXI4	AXI5
Outstanding Transactions	Up to 16	Up to 256
Atomic Operations	Not supported	AtomicStore, AtomicLoad, AtomicSwap, AtomicCompare
Memory Tagging	Not supported	AWMEMATTR, ARMEMATTR
Cache Maintenance	Limited	Enhanced CMO support
QoS	ARQOS/AWQOS	Enhanced QoS with ARQOSACCEPT/AWQOSACCEPT
User Signals	ARUSER/AWUSER/ etc.	Extended user signal widths
Chunking	Not supported	AWCHUNKEN for large transfers

Feature	AXI4	AXI5
Loop Support	Not supported	AWLOOP, ARLOOP for DMA
Poison	Not supported	RPOISON, WPOISON for error propagation

9.3 Module Categories

9.3.1 Master Components

Module	Description	Documentation	Status
axi5_master_rd	AXI5 read master with skid buffers and burst support	axi5_master_rd.md	Documented
axi5_master_rd_cg	Clock-gated AXI5 read master	axi5_master_rd_cg.md	Documented
axi5_master_wr	AXI5 write master with address/data coordination	axi5_master_wr.md	Documented
axi5_master_wr_cg	Clock-gated AXI5 write master	axi5_master_wr_cg.md	Documented

9.3.2 Slave Components

Module	Description	Documentation	Status
axi5_slave_rd	AXI5 read slave with configurable response handling	axi5_slave_rd.md	Documented
axi5_slave_rd_cg	Clock-gated AXI5 read slave	axi5_slave_rd_cg.md	Documented
axi5_slave_wr	AXI5 write slave with write response generation	axi5_slave_wr.md	Documented
axi5_slave_wr_cg	Clock-gated AXI5 write slave	axi5_slave_wr_cg.md	Documented

9.3.3 Monitor Components

Module	Description	Documentation	Status
axi5_master_rd_mon	AXI5 read master with integrated transaction monitor	axi5_master_rd_mon.md	Documented
axi5_master_rd_mon_cg	Clock-gated read master with monitor	axi5_master_rd_mon_cg.md	Documented
axi5_master_wr_mon	AXI5 write master with integrated transaction monitor	axi5_master_wr_mon.md	Documented
axi5_master_wr_mon_cg	Clock-gated write master with monitor	axi5_master_wr_mon_cg.md	Documented
axi5_slave_rd_mon	AXI5 read slave with integrated transaction monitor	axi5_slave_rd_mon.md	Documented
axi5_slave_rd_mon_cg	Clock-gated read slave with monitor	axi5_slave_rd_mon_cg.md	Documented
axi5_slave_wr_mon	AXI5 write slave with integrated transaction monitor	axi5_slave_wr_mon.md	Documented
axi5_slave_wr_mon_cg	Clock-gated write slave with monitor	axi5_slave_wr_mon_cg.md	Documented

9.4 Key Features

9.4.1 AXI5 Protocol Support

- **Full AXI5 Compliance:** Complete AMBA 5 AXI protocol implementation
- **Burst Support:** INCR, FIXED, and WRAP burst types
- **Outstanding Transactions:** Up to 256 concurrent transactions
- **ID Reordering:** Full transaction ID support with reordering
- **Atomic Operations:** AtomicStore, AtomicLoad, AtomicSwap, AtomicCompare

9.4.2 Advanced Features

- **Memory Tagging:** AWMEMATTR/ARMEMATTR for memory type attributes
- **QoS Enhancement:** Extended quality of service signaling
- **Poison Support:** RPOISON/WPOISON for error propagation
- **Chunking:** AWCHUNKEN for efficient large transfers
- **Loop Operations:** AWLOOP/ARLOOP for DMA acceleration

9.4.3 Power Management

- **Clock Gating:** Per-module clock gating for power reduction
- **Idle Detection:** Automatic clock gate when interface is idle
- **Low-Power Modes:** Support for system low-power state integration

9.4.4 Monitoring and Debug

- **Integrated Monitors:** Combined master/slave + monitor modules
 - **Transaction Tracking:** Complete transaction lifecycle monitoring
 - **Performance Metrics:** Bandwidth, latency, and utilization
 - **64-bit Monitor Bus:** Standardized packet format
-

9.5 Quick Start

9.5.1 Using AXI5 Read Master

```
axi5_master_rd #(
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4),
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(64),
    .AXI_DATA_WIDTH(128),
    .AXI_USER_WIDTH(4)
) u_axi5_rd_master (
    .aclk          (clk),
    .aresetn       (resetn),

    // FUB (Functional Unit Block) interface
    .fub_axi_arid   (fub_arid),
    .fub_axi_araddr  (fub_araddr),
    .fub_axi_arlen   (fub_arlen),
    .fub_axi_arsize  (fub_arsize),
    .fub_axi_arburst  (fub_arburst),
    .fub_axi_arlock  (fub_arlock),
    .fub_axi_arcache  (fub_arcache),
```

```

.fub_axi_arprot      (fub_arprot),
.fub_axi_arqos       (fub_arqos),
.fub_axi_arregion    (fub_arregion),
.fub_axi_aruser       (fub_aruser),
.fub_axi_arvalid      (fub_arvalid),
.fub_axi_arready      (fub_arready),

.fub_axi_rid         (fub_rid),
.fub_axi_rdata        (fub_rdata),
.fub_axi_rresp        (fub_rresp),
.fub_axi_rlast        (fub_rlast),
.fub_axi_ruser        (fub_ruser),
.fub_axi_rvalid       (fub_rvalid),
.fub_axi_rready       (fub_rready),

// AXI5 master interface
.m_axi_ar*            (mem_ar*),
.m_axi_r*             (mem_r*)
);

```

9.5.2 Using AXI5 Write Master

```

axi5_master_wr #(
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(4),
    .SKID_DEPTH_B(2),
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(64),
    .AXI_DATA_WIDTH(128),
    .AXI_USER_WIDTH(4)
) u_axi5_wr_master (
    .aclk                (clk),
    .aresetn              (resetn),

    // FUB interface
    .fub_axi_awid         (fub_awid),
    .fub_axi_awaddr        (fub_awaddr),
    .fub_axi_awlen         (fub_awlen),
    .fub_axi_awsz         (fub_awsz),
    .fub_axi_awburst       (fub_awburst),
    .fub_axi_awlock        (fub_awlock),
    .fub_axi_awcache       (fub_awcache),
    .fub_axi_awprot        (fub_awprot),
    .fub_axi_awqos         (fub_awqos),
    .fub_axi_awregion      (fub_awregion),
    .fub_axi_awuser        (fub_awuser),
    .fub_axi_awvalid       (fub_awvalid),
    .fub_axi_awready       (fub_awready),

```

```

.fub_axi_wdata      (fub_wdata),
.fub_axi_wstrb      (fub_wstrb),
.fub_axi_wlast      (fub_wlast),
.fub_axi_wuser       (fub_wuser),
.fub_axi_wvalid      (fub_wvalid),
.fub_axi_wready      (fub_wready),

.fub_axi_bid         (fub_bid),
.fub_axi_bresp       (fub_bresp),
.fub_axi_buser       (fub_buser),
.fub_axi_bvalid      (fub_bvalid),
.fub_axi_bready      (fub_bready),

// AXI5 master interface
.m_axi_aw*           (mem_aw*),
.m_axi_w*            (mem_w*),
.m_axi_b*            (mem_b*)
);

9.5.3 Using Integrated Monitor Modules
// AXI5 master with built-in transaction monitoring
axi5_master_rd_mon #(
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(64),
    .AXI_DATA_WIDTH(128)
) u_axi5_rd_master_mon (
    .aclk              (clk),
    .aresetn           (resetn),

// FUB and AXI interfaces (same as non-monitor version)
    .fub_axi_ar*       (...),
    .fub_axi_r*        (...),
    .m_axi_ar*         (...),
    .m_axi_r*          (...),

// Monitor bus output
    .mon_valid         (rd_mon_valid),
    .mon_ready         (rd_mon_ready),
    .mon_data          (rd_mon_data)
);

```

9.6 Testing

All AXI5 modules are verified using CocoTB-based testbenches located in `val/amba/`:

```
# Run all AXI5 tests
```

```
pytest val/amba/test_axi5*.py -v
```

```
# Run specific module tests
```

```
pytest val/amba/test_axi5_master_rd.py -v
```

```
pytest val/amba/test_axi5_master_wr.py -v
```

```
pytest val/amba/test_axi5_slave_rd.py -v
```

```
pytest val/amba/test_axi5_slave_wr.py -v
```

9.7 Protocol Details

9.7.1 AXI5 Channel Overview

AXI5 maintains the five-channel architecture from AXI4:

Channel	Direction	Purpose
AR (Address Read)	Master to Slave	Read address and control
R (Read Data)	Slave to Master	Read data and response
AW (Address Write)	Master to Slave	Write address and control
W (Write Data)	Master to Slave	Write data
B (Write Response)	Slave to Master	Write response

9.7.2 AXI5 Enhanced Signals

Signal	Channel	Description
AWATOP	AW	Atomic operation type
AWMEMATTR	AW	Memory attributes for tagging
ARMEMATTR	AR	Memory attributes for tagging
AWCHUNKEN	AW	Chunking enable for

Signal	Channel	Description
		large transfers
AWLOOP	AW	Loop iteration count
ARLOOP	AR	Loop iteration count
RPOISON	R	Data poisoned indicator
WPOISON	W	Write data poisoned indicator
AWQOSACCEPT	B	QoS acceptance indicator
ARQOSACCEPT	R	QoS acceptance indicator

9.7.3 Burst Types

ARBURST/AWBURST	Type	Description
2'b00	FIXED	Fixed address for FIFO access
2'b01	INCR	Incrementing address burst
2'b10	WRAP	Wrapping burst for cache line
2'b11	Reserved	Not used

9.8 Design Notes

9.8.1 FUB Interface Pattern

All AXI5 modules use the FUB (Functional Unit Block) interface pattern: - **fub_axi_*** signals connect to internal logic - **m_axi_*** or **s_axi_*** signals connect to external AXI bus - Skid buffers between FUB and external interfaces for timing closure

9.8.2 Migration from AXI4

AXI5 modules are backward compatible with AXI4: - Core protocol unchanged (5-channel architecture) - New signals can be tied to default values - AWATOP tied to 0 disables atomic operations - AWMEMATTR/ARMEMATTR can use default values - User signal widths can match AXI4 configurations

9.8.3 Monitor Integration

The *_mon variants combine master/slave with integrated monitor: - Single instantiation for both data path and monitoring - Shared clock and reset - Monitor bus output for transaction visibility - Reduced resource usage vs. separate monitor instantiation

9.9 Performance Characteristics

Metric	Typical Value
Maximum Frequency	500+ MHz
Latency (simple transaction)	1-2 clock cycles
Throughput (128-bit, 500 MHz)	64 GB/s per direction
Outstanding Transactions	Up to 256

9.10 Related Documentation

- [AXI4 Modules](#) - AXI4 protocol components
 - [APB5 Modules](#) - APB5 protocol components
 - [AXI5 Modules](#) - AXI5-Stream components
 - [GAXI Modules](#) - Generic AXI utilities
 - [Shared Infrastructure](#) - Common components
-

9.11 References

9.11.1 Specifications

- ARM AMBA 5 AXI Protocol Specification
- ARM AMBA AXI and ACE Protocol Specification (AXI4)

9.11.2 Source Code

- RTL: rtl/amba/axi5/
 - Tests: val/amba/test_axi5*.py
 - Framework: bin/TBClasses/components/axi4/
-

Last Updated: 2025-12-26

9.12 Navigation

- [Back to RTLamba Index](#)
- [Back to Main Documentation Index](#)